

CUDA Kernel 面试背题笔记

本书整理自 LeetCUDA 项目，系统汇编了 CUDA 面试中最高频的 GPU Kernel 实现，涵盖从 Warp/Block 原语、Elementwise、Softmax、GEMM、FlashAttention 等面试核心知识体系。

每个 Kernel 配备详细的中文面试要点注释 (WHY + HOW)，并以递进式优化路线 (naive → tiling → vectorize → tensor core → warp specialized) 展示 GPU 性能调优思维。

26 个 Kernel · 10 个 Phase · 全中文注释

项目主页: <https://github.com/xlite-dev/LeetCUDA>

更新日期: 2026 年 6 月 18 日

```

1 // =====
2 // notes-v2.cu — CUDA Kernel 面试题笔记
3 // =====
4 //
5 // 整理自 LeetCUDA 项目 (https://github.com/xlite-dev/LeetCUDA) , 涵盖:
6 //   - 面试高频 CUDA kernel 的完整实现 (共 26 个 kernel)
7 //   - 每类 kernel 附带详细的面试要点注释 (WHY + HOW)
8 //   - 优化技术的递进式讲解 (naive → tiling → vectorize → tensor core → ws)
9 //   - BLAS 语义: N=col-major(Normal), T=row-major(Transposed)
10 //
11 // 10 个 Phase 覆盖:
12 //   Phase 0 — 面试框架速查 (GPU 架构 / Memory Hierarchy / Roofline / 优化清单)
13 //   Phase 1 — 基础原语: Warp Reduce / Block Reduce (含 broadcast 增强版)
14 //   Phase 2 — Elementwise: ReLU / Elementwise Add (基础 + float4 向量化)
15 //   Phase 3 — Softmax: naive → safe → online + RMS/Layer Norm
16 //   Phase 4 — GEMV: SGEMV K32/K128/K16 (warp-per-row)
17 //   Phase 5 — GEMM *: SGEMM → HGEMM → MMA m16n8k16(TN布局) → WGMMMA m64n128k16
18 //   Phase 6 — RoPE: 旋转位置编码 (Llama 风格 theta=10000)
19 //   Phase 7 — Mat Transpose: 基础版 + BCF merge_write 最佳版 (Bank Conflict专题)
20 //   Phase 8 — 杂项: Dot Product / Block Reduce All / Histogram
21 //   Phase 9 — FlashAttention split_q (FA-2, 含 online softmax + P@V 寄存器复用)
22 //
23 // =====
24 // Phase 0: 面试框架速查 (纯注释, 面试开场必备的基础知识)
25 // =====
26
27 // ---- GPU 架构速查 ----
28 //
29 // SM (Streaming Multiprocessor) 内部结构:
30 //   - Warp Scheduler ×4: 每 SM 4 个 warp scheduler, 每个每周期可发射 1 条指令
31 //   - Register File: 每 SM 65536 × 32-bit = 256KB
32 //   - Shared Memory / L1: 可配置, 最大 shared memory ~228KB (Hopper)
33 //   - Tensor Cores: Hopper 每 SM 4 个; Blackwell 数量随型号/定义不同, 建议以官方 ISV guide 为准
34 //   - Warp = 32 threads: 最小调度单元, SIMT 执行模型
35 //
36 // Memory Hierarchy 带宽数量级 (H100 参考):
37 //   HBM3: ~3.35 TB/s (理论), 实际 ~2.5-3.0 TB/s
38 //   L2 Cache: ~12 TB/s (50MB, 跨 SM 共享)
39 //   L1/SMEM: ~19 TB/s (每 SM ~228KB)
40 //   Register: ~0 延迟, ~100+ TB/s 等效带宽
41 //
42 // 关键瓶颈判断:
43 //   Memory-bound: AI (Arithmetic Intensity) < 机器 FLOPS/带宽 比值
44 //   Compute-bound: AI 足够大, 受限于计算吞吐
45 //   Latency-bound: 线程不够多, 无法隐藏内存延迟
46 //
47 // Occupancy 公式:
48 //   occupancy = active_warps / max_warps_per_SM
49 //   受三类资源分别取下限: 每线程寄存器数 → threads/SM; 每 block shared memory → blocks/SM; block 大小 → blocks/SM
50
51 // ---- 常见优化手段速查清单 ----
52 //
53 // 1. Coalesced Memory Access (合并访问)
54 //   - 同一 warp 的线程访问连续的 128B 对齐地址 → 1 次内存事务
55 //   - 否则产生多次事务 (最坏 32 次)
56 //
57 // 2. Tiling (分块)
58 //   - 将数据从 HBM 分块加载到 shared memory 复用, 减少 HBM 访问
59 //   - GEMM: Block Tile (BM×BN) + K Tile (BK)
60 //
61 // 3. Vectorized Memory Access (向量化)
62 //   - 使用 float4/half2 等向量类型, 减少 load/store 指令数
63 //   - float4 = 128-bit, 单条指令加载 16 bytes
64 //
65 // 4. Thread Tile (寄存器分块)
66 //   - 每个线程计算多个输出元素 (TM×TN), 提高计算密度
67 //   - 减少线程总数, 降低同步开销
68 //
69 // 5. Bank Conflict Avoidance
70 //   - Shared memory 有 32 banks × 4 bytes

```

```

71 // - 同 warp 多线程访问同一 bank 的不同地址 → bank conflict → 串行化
72 // - 解决方案: PAD (在每行末尾加 1 个元素打破对齐)
73 //
74 // 6. Pipeline / Double Buffering (流水线)
75 // - cp.async 异步拷贝下一批数据时同时做当前批的计算
76 // - Stage 数 = 2/3/4, 权衡 shared memory 占用和延迟隐藏
77 //
78 // 7. Tensor Core (MMA / WGMMMA)
79 // - MMA m16n8k16 (Ampere): warp 级指令, 单 warp 完成 16×8×16 的矩阵乘
80 // - WGMMMA m64n128k16 (Hopper): warpgroup 级指令 (128 threads), 异步执行
81 //
82 // 8. Warp Specialization (Hopper+)
83 // - Producer warpgroup 做 TMA 数据搬运, Consumer warpgroup 做计算
84 // - 通过 cuda::barrier 同步, 完全解耦数据搬运和计算
85 //
86 // 9. TMA (Tensor Memory Accelerator, Hopper+)
87 // - 硬件 DMA 引擎, 支持 2D/3D 寻址, 零寄存器开销
88 // - 配合 cp.async.bulk 实现异步数据搬运
89
90 // ---- Roofline 分析公式 ----
91 //
92 // AI (Arithmetic Intensity) = FLOPs / Bytes_transferred
93 //
94 // GEMM (M=N=K=4096):
95 // FLOPs = 2 × M × N × K = 2 × 40963 ≈ 137 GFLOPS
96 // Bytes = (M×K + K×N + M×N) × sizeof(float) ≈ 200 MB
97 // AI ≈ 137G / 200M ≈ 685 FLOPS/Byte → compute-bound (远超 H100 ridge point:
98 // FP16 TC ≈ 295:1, FP32 ≈ 20:1)
99 //
100 // GEMV (M=4096, K=4096):
101 // FLOPs = 2 × M × K = 2 × 40962 ≈ 33 MFLOPS
102 // Bytes = (M×K + K + M) × sizeof(float) ≈ 67 MB
103 // AI ≈ 33M / 67M ≈ 0.5 FLOPS/Byte → severely memory-bound
104 //
105 // Softmax (N=4096): AI ≈ (5×N) / (2×N×4) = 5/8 ≈ 0.625 FLOPS/Byte → memory-bound
106
107 // =====
108 // Phase 1: 头文件 + 宏定义 + 基础原语 (Warp Reduce / Block Reduce)
109 // =====
110 // 面试要点:
111 // - warp_reduce: 用 __shfl_xor_sync 做蝶形归约, O(logN) 步, 无需 shared
112 // memory
113 // - block_reduce: 两级归约 (warp → shared memory → warp0 broadcast),
114 // 注意最后必须 broadcast 回所有线程 (__shfl_sync), 否则只有 warp0 知道结果
115 // - 为什么不用 __shfl_down_sync? xor 模式所有线程做相同工作量, 更均衡
116
117 #include <algorithm>
118 #include <cuda_fp16.h>
119 #include <cuda_runtime.h>
120 #include <float.h>
121 #include <stdio.h>
122 #include <stdlib.h>
123
124 #define WARP_SIZE 32
125 #define INT4(value) (reinterpret_cast<int4*>(&(value)))[0])
126 #define FLOAT4(value) (reinterpret_cast<float4*>(&(value)))[0])
127 #define HALF2(value) (reinterpret_cast<half2*>(&(value)))[0])
128
129 // =====
130 // Phase 1a: Warp Reduce (warp 内归约, 纯寄存器操作, 无需 shared memory)
131 // =====
132
133 // Warp Reduce Sum — generic (used by both FP32 and FP16 contexts)
134 // 使用 __shfl_xor_sync 做蝶形归约 (butterfly reduction)
135 // 复杂度 O(logN), N=32 时仅需 5 步
136 // 模板参数: T=数据类型, kWarpSize=segment width (默认 32)
137 // kWarpSize 会作为 __shfl_xor_sync 的第 4 个实参 width, 限制 shuffle 在同一 segment 内
138 // 当 kWarpSize < 32 (如 FA 中 kWarpSize=4) 时, 只有同 segment 的 lane 参与通信
139 // 蝶形归约示意 (以 warpSize=8 为例, 实际 warpSize=32 有 5 次迭代):
140 //

```

```

141 // 初始: 每个 lane 持有自己的值 v0..v7
142 // lane:  0    1    2    3    4    5    6    7
143 // val:  v0   v1   v2   v3   v4   v5   v6   v7
144 //
145 // mask=4 (第1次迭代, lane i 与 lane i^4 交换并累加):
146 //
147 // 对:  (0,4) (1,5) (2,6) (3,7)
148 //
149 // lane:  0    1    2    3    4    5    6    7
150 // val: v0+v4 v1+v5 v2+v6 v3+v7 v4+v0 v5+v1 v6+v2 v7+v3
151 //
152 // mask=2 (第2次迭代, lane i 与 lane i^2 交换并累加):
153 //
154 // 对:  (0,2) (1,3) (4,6) (5,7)
155 //      —— 前4个一组 ——      —— 后4个一组 ——
156 //
157 // lane:  0    1    2    3    4    5    6    7
158 // val:  $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$   $\Sigma\{0,2,4,6\}$   $\Sigma\{1,3,5,7\}$  (每Lane持有前一轮2个值的和再
    加本轮配对)
159 //      = v0+v2+v4+v6 ... (逐步归约)
160 //
161 // mask=1 (第3次迭代, lane i 与 lane i^1 交换并累加):
162 //
163 // 对:  (0,1) (2,3) (4,5) (6,7)
164 //
165 // lane:  0    1    2    3    4    5    6    7
166 // val:  $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$   $\Sigma\text{all}$  ← 所有 lane 拥有全归约结果!
167 //
168 // mask=0: 循环终止, 归约完成。
169 //
170 // 关键性质:
171 // - XOR 配对是对称的: lane i 的配对对象是 lane i^mask, 而 (i^mask)^mask = i
172 // - 每轮每个 lane 只和恰好 1 个其他 lane 通信 (一对一, 无冲突)
173 // - 每轮信息传递距离减半: 16→8→4→2→1 (距离减半, 信息翻倍)
174 // -  $O(\log_2 N)$  步完成, 无需 shared memory, 纯寄存器操作
175 // - __shfl_xor_sync 第四个参数 kWarpSize 限制 segment width:
176 //   当 kWarpSize=4 时, 只有同 segment(4个一组)内的 lane 参与 shuffle
177 template <typename T = float, const int kWarpSize = WARP_SIZE>
178 __device__ __forceinline__ T warp_reduce_sum(T val) {
179 #pragma unroll
180     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
181         val += __shfl_xor_sync(0xffffffff, val, mask, kWarpSize);
182     }
183     return val;
184 }
185
186 // Warp Reduce Max — generic
187 template <typename T = float, const int kWarpSize = WARP_SIZE>
188 __device__ __forceinline__ T warp_reduce_max(T val) {
189 #pragma unroll
190     for (int mask = kWarpSize >> 1; mask >= 1; mask >>= 1) {
191         val = max(val, __shfl_xor_sync(0xffffffff, val, mask, kWarpSize));
192     }
193     return val;
194 }
195
196 // =====
197 // Phase 1b: Block Reduce (block 内归约, 两级: warp → shared memory → warp reduce)
198 // =====
199
200 // Block Reduce Sum — FP32 (增强版, 带 broadcast)
201 // 两级归约流程:
202 // 1. 每个 warp 内做 warp_reduce_sum → 得到每 warp 的一个值
203 // 2. warp leader (lane=0) 写入 shared memory
204 // 3. syncthreads 后, lane 0~NUM_WARPS-1 读取 shared memory
205 // 4. 所有 warp 内再做一次 warp_reduce<NUM_WARPS> → 得到最终结果
206 // 5. __shfl_sync broadcast 到所有线程 (关键! 否则每个warp只有lane<NUM_WARPS 知道结果)
207 template <const int NUM_THREADS = 256>
208 __device__ float block_reduce_sum(float val) {
209     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
210     int warp = threadIdx.x / WARP_SIZE;

```

```

211 int lane = threadIdx.x % WARP_SIZE;
212 static __shared__ float shared[NUM_WARPS];
213
214 float value = warp_reduce_sum<WARP_SIZE>(val);
215 if (lane == 0)
216     shared[warp] = value;
217 __syncthreads();
218 value = (lane < NUM_WARPS) ? shared[lane] : 0.0f;
219 value = warp_reduce_sum<NUM_WARPS>(value);
220 // 关键: broadcast 结果到所有线程, 后续用 result
221 // 做除法等操作时所有线程都能拿到
222 value = __shfl_sync(0xffffffff, value, 0, 32);
223 return value;
224 }
225
226 // Block Reduce Max — FP32 (增强版, 带 broadcast)
227 template <const int NUM_THREADS = 256>
228 __device__ float block_reduce_max(float val) {
229     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
230     int warp = threadIdx.x / WARP_SIZE;
231     int lane = threadIdx.x % WARP_SIZE;
232     static __shared__ float shared[NUM_WARPS];
233
234     float value = warp_reduce_max<WARP_SIZE>(val);
235     if (lane == 0)
236         shared[warp] = value;
237     __syncthreads();
238     value = (lane < NUM_WARPS) ? shared[lane] : -FLT_MAX;
239     value = warp_reduce_max<NUM_WARPS>(value);
240     value = __shfl_sync(0xffffffff, value, 0, 32);
241     return value;
242 }
243
244 // =====
245 // Phase 2: Elementwise Ops (逐元素操作, 演示 coalesced access + vectorize)
246 // =====
247 // 面试要点:
248 //   - 逐元素操作是最简单的 kernel, 核心考点是 memory coalescing
249 //   - float4 向量化可将内存事务数减为 1/4, 大幅提升 bandwidth utilization
250 //   - grid/block 维度设计: grid(N/threads), block(threads), 一维即可
251
252 // ---- ReLU: y = max(0, x) ----
253 // Grid: ((N + 255) / 256, 1, 1)
254 // Block: (256, 1, 1)
255 // source: LeetCode/kernels/relu/relu.cu
256 __global__ void relu(float *x, float *y, int N) {
257     int idx = blockIdx.x * blockDim.x + threadIdx.x;
258     if (idx < N)
259         y[idx] = fmaxf(0.0f, x[idx]);
260 }
261
262 // ReLU + float4 向量化: 每个线程处理 4 个元素, 减少 75% 的 load/store 指令
263 // block(64)×4(float4)=256 元素/block, 与基础版吞吐相同
264 // Grid: ((N + 255) / 256, 1, 1)
265 // Block: (64, 1, 1)
266 // 注意: 该版本默认地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
267 // source: LeetCode/kernels/relu/relu.cu
268 __global__ void relu_vec4(float *x, float *y, int N) {
269     int idx = (blockIdx.x * blockDim.x + threadIdx.x) * 4;
270     if (idx < N) {
271         float4 reg_x = FLOAT4(x[idx]); // 单条 128-bit load
272         float4 reg_y;
273         reg_y.x = fmaxf(0.0f, reg_x.x);
274         reg_y.y = fmaxf(0.0f, reg_x.y);
275         reg_y.z = fmaxf(0.0f, reg_x.z);
276         reg_y.w = fmaxf(0.0f, reg_x.w);
277         FLOAT4(y[idx]) = reg_y; // 单条 128-bit store
278     }
279 }

```

```

281 // ---- Elementwise Add: c = a + b ----
282 // Grid: ((N + 255) / 256, 1, 1)
283 // Block: (256, 1, 1)
284 // source: LeetCUDA/kernels/elementwise/elementwise.cu
285 __global__ void elementwise_add(float *a, float *b, float *c, int N) {
286     int idx = blockIdx.x * blockDim.x + threadIdx.x;
287     if (idx < N)
288         c[idx] = a[idx] + b[idx];
289 }
290
291 // Elementwise Add + float4 向量化
292 // Grid: ((N + 255) / 256, 1, 1)
293 // Block: (64, 1, 1), block(64)×4(float4)=256 元素/block
294 // 注意: 主路径要求 4 元素对齐; 尾部不足 4 个元素时回退到标量处理
295 // source: LeetCUDA/kernels/elementwise/elementwise.cu
296 __global__ void elementwise_add_vec4(float *a, float *b, float *c, int N) {
297     int idx = 4 * (blockIdx.x * blockDim.x + threadIdx.x);
298     if ((idx + 3) < N) {
299         float4 reg_a = FLOAT4(a[idx]);
300         float4 reg_b = FLOAT4(b[idx]);
301         float4 reg_c;
302         reg_c.x = reg_a.x + reg_b.x;
303         reg_c.y = reg_a.y + reg_b.y;
304         reg_c.z = reg_a.z + reg_b.z;
305         reg_c.w = reg_a.w + reg_b.w;
306         FLOAT4(c[idx]) = reg_c;
307     } else if (idx < N) {
308         for (int i = 0; (idx + i) < N; i++) {
309             c[idx + i] = a[idx + i] + b[idx + i];
310         }
311     }
312 }
313
314 // =====
315 // Phase 3: Reduce 类 Ops — Softmax / RMS Norm / Layer Norm
316 // =====
317 // 面试要点:
318 //   - Softmax 三种实现递进: naive (溢出) → safe (2-pass, max 减法) →
319 //     online (1-pass, 增量更新)
320 //   - Online Softmax 是 FlashAttention 的数学基础
321 //   - RMS Norm vs Layer Norm: RMS 只需 1 次 reduce, Layer Norm 需要 2 次 (mean
322 //     + variance)
323 //   - Per-token 设计: 一个 block 处理一个 token, 无需跨 block 同步
324
325 // ---- Online Softmax 辅助结构 ----
326 // MD struct: 存储 running max (m) 和 running denominator (d)
327 // 算法来源: "Online normalizer calculation for softmax" (arXiv:1805.02867)
328 // 核心递推公式:
329 //   m_new = max(m_old, x_i)
330 //   d_new = d_old * exp(m_old - m_new) + exp(x_i - m_new)
331 struct __align__(8) MD {
332     float m; // running max
333     float d; // running denominator (sum of exp(x - max))
334 };
335
336 // Warp Reduce for Online Softmax
337 // 与普通 reduce 不同: 归约时需同时更新 m 和 d (因为 max 在不断变大)
338 template <const int kWarpSize = WARP_SIZE>
339 __device__ __forceinline__ MD warp_reduce_md_op(MD value) {
340     unsigned int mask = 0xffffffff;
341 #pragma unroll
342     for (int stride = kWarpSize >> 1; stride >= 1; stride >>= 1) {
343         MD other;
344         other.m = __shfl_xor_sync(mask, value.m, stride);
345         other.d = __shfl_xor_sync(mask, value.d, stride);
346
347         bool value_bigger = (value.m > other.m);
348         MD bigger_m = value_bigger ? value : other;
349         MD smaller_m = value_bigger ? other : value;

```

```

351 // 关键: 更新 d 时需要 rescale 旧的 d 到新的 max 尺度下
352 value.d = bigger_m.d + smaller_m.d * __expf(smaller_m.m - bigger_m.m);
353 value.m = bigger_m.m;
354 }
355 return value;
356 }
357
358 // =====
359 // Phase 3a: Softmax — 三级递进 (面试核心考点)
360 // =====
361 // 面试常问: 「Softmax 有哪些实现方式? 各有什么优缺点?」
362 // 回答线索: naive(溢出) → safe → online
363
364 // ---- Level 1: 基础 Softmax (per-token, 无 max 减法, 数值不稳定) ----
365 // grid(S*h/h, h), block(h), 一个 block 处理一个 token
366 // 问题: x 值很大时 exp(x) 溢出为 inf
367 template <const int NUM_THREADS = 256>
368 // Grid: (S, 1, 1), S=batch*seq_len, DISPATCH_SOFTMAX_F32_PER_TOKEN_KERNEL
369 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024, 一个 block 处理一个 token
370 // source: LeetCUDA/kernels/softmax/softmax.cu
371 __global__ void softmax_per_token(float *x, float *y, int N) {
372     const int tid = threadIdx.x;
373     const int idx = blockIdx.x * blockDim.x + tid;
374
375     float exp_val = (idx < N) ? expf(x[idx]) : 0.0f;
376     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
377     if (idx < N)
378         y[idx] = exp_val / exp_sum;
379 }
380
381 // ---- Level 2: Safe Softmax (2-pass: 先 max 再 exp, 数值稳定) ----
382 // 面试重点: 为什么 Softmax 需要 Safe?
383 //   - expf 溢出阈值约 88.7: exp(88)≈1.65e38 (接近 float32 上限 3.4e38), exp(89)≈4.5e38 已溢出为 inf
384 //   - 减去 max 后: exp(x - max) ≤ exp(0) = 1.0, 永不超过 1
385 //   - 数学等价性: softmax(x) = softmax(x - c) 对任意常数 c 成立
386 //   - 代价: 2 次 block reduce (先 max, 再 sum), 但仍 O(N/B) 高效
387 template <const int NUM_THREADS = 256>
388 // Grid: (S, 1, 1)
389 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
390 // source: LeetCUDA/kernels/softmax/softmax.cu
391 __global__ void safe_softmax_per_token(float *x, float *y, int N) {
392     const int tid = threadIdx.x;
393     const int idx = blockIdx.x * blockDim.x + tid;
394
395     // Pass 1: block reduce max — 找最大值
396     float val = (idx < N) ? x[idx] : -FLT_MAX;
397     float max_val = block_reduce_max<NUM_THREADS>(val);
398
399     // Pass 2: exp(x - max) → block reduce sum
400     float exp_val = (idx < N) ? expf(x[idx] - max_val) : 0.0f;
401     float exp_sum = block_reduce_sum<NUM_THREADS>(exp_val);
402
403     if (idx < N)
404         y[idx] = exp_val / exp_sum;
405 }
406
407 // ---- Level 3: Online Safe Softmax (FlashAttention 的数学基础) ----
408 // 面试重点: Online Softmax 为什么重要?
409 //   - Safe Softmax 需要 3 遍遍历: 找 max → exp+sum → 除法
410 //   - Online Softmax 用递推公式合并为 1 遍遍历
411 //   - 核心思想: 处理新元素时, 用新旧 max 的差值 rescale 旧 denominator
412 //   - 正是 FlashAttention 中 "online rescaling":
413 //       Q_new = diag(exp(m_old - m_new)) * Q_old + exp(m_cur - m_new) * P@V
414 // 算法参考: "Online normalizer calculation for softmax" (arXiv:1805.02867)
415 // 注意: 这里默认一个 block 处理一个 token; 边界线程的 d=0 只参与归约, 不会写回 y
416 template <const int NUM_THREADS = 256>
417 // Grid: (S, 1, 1)
418 // Block: (H, 1, 1), 由外层 dispatch 选择 H=32/64/128/256/512/1024
419 // source: LeetCUDA/kernels/softmax/softmax.cu
420 __global__ void online_safe_softmax_per_token(const float *x, float *y, int N) {

```

```

421 int local_tid = threadIdx.x;
422 int global_tid = blockIdx.x * NUM_THREADS + threadIdx.x;
423 const int WARP_NUM = NUM_THREADS / WARP_SIZE;
424 int warp_id = local_tid / WARP_SIZE;
425 int lane_id = local_tid % WARP_SIZE;
426
427 // 初始化: 每个线程持有一个 (max, denom) 对
428 MD val;
429 val.m = global_tid < N ? x[global_tid] : -FLT_MAX;
430 val.d = global_tid < N ? 1.0f : 0.0f;
431
432 // Block reduce: warp_reduce_md_op 在归约中自动更新 m 和 d
433 __shared__ MD shared[WARP_NUM];
434 MD res = warp_reduce_md_op<WARP_SIZE>(val);
435
436 if (lane_id == 0)
437     shared[warp_id] = res;
438 __syncthreads();
439
440 // 第二级归约: warp0 收集各 warp 结果再做一次 md_op (复用 block_reduce 模式)
441 // 只有 local_tid < 32 的线程参与; shared[0] 在第二次 __syncthreads 后才对整个 block 可见
442 if (local_tid < WARP_SIZE) {
443     MD block_res =
444         local_tid < WARP_NUM ? shared[local_tid] : MD{-FLT_MAX, 0.0f};
445     block_res = warp_reduce_md_op<WARP_NUM>(block_res);
446     if (local_tid == 0) {
447         shared[0] = block_res;
448     }
449 }
450 __syncthreads();
451
452 // 用全局 max 和 denom 做最终 softmax
453 MD final_res = shared[0];
454 float d_total_inverse = __fdivdef(1.0f, final_res.d);
455 // 边界线程即使看到 d=0 的填充值, 也不会走到写回路径
456 if (global_tid < N) {
457     y[global_tid] = __expf(x[global_tid] - final_res.m) * d_total_inverse;
458 }
459 }
460
461 // =====
462 // Phase 3b: RMS Normalization (1-pass reduce)
463 // =====
464 // 面试要点:
465 //   - RMS Norm:  $y = (x / \text{rms}(x)) * g$ ,  $1/\text{rms}(x) = \text{rsqrt}(\text{mean}(x^2))$ 
466 //   - 只需 1 次 block reduce (sum of squares), 比 Layer Norm 少 1 次同步
467 //   - Llama 系列使用 RMS Norm
468 //   - grid(N, K/K), block(K): 一行一个 block
469
470 template <const int NUM_THREADS = 128>
471 // Grid: (N, 1, 1), N=batch*seq_len, 每行一个 block
472 // Block: (128, 1, 1), NUM_THREADS=K=128 (K>128 时调整模板参数)
473 // source: LeetCode/kernels/rms-norm/rms_norm.cu
474 __global__ void rms_norm(float *x, float *y, float g, int N, int K) {
475     int tid = threadIdx.x;
476     int bid = blockIdx.x;
477     int idx = bid * blockDim.x + threadIdx.x;
478     const float epsilon = 1e-5f;
479
480     __shared__ float s_variance;
481     float value = (idx < N * K) ? x[idx] : 0.0f;
482     float variance = value * value;
483     variance = block_reduce_sum<NUM_THREADS>(variance);
484     if (tid == 0)
485         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/rms(x)
486     __syncthreads();
487     if (idx < N * K)
488         y[idx] = (value * s_variance) * g;
489 }

```



```

491 // RMS Norm + float4
492 template <const int NUM_THREADS = 128 / 4>
493 // Grid: (N, 1, 1)
494 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
495 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
496 // source: LeetCode/kernels/rms-norm/rms_norm.cu
497 __global__ void rms_norm_vec4(float *x, float *y, float g, int N, int K) {
498     int tid = threadIdx.x;
499     int bid = blockIdx.x;
500     int idx = (bid * blockDim.x + threadIdx.x) * 4;
501     const float epsilon = 1e-5f;
502
503     __shared__ float s_variance;
504     float4 reg_x = FLOAT4(x[idx]);
505     float variance = (idx < N * K) ? (reg_x.x * reg_x.x + reg_x.y * reg_x.y +
506                                     reg_x.z * reg_x.z + reg_x.w * reg_x.w)
507                                     : 0.0f;
508     variance = block_reduce_sum<NUM_THREADS>(variance);
509     if (tid == 0)
510         s_variance = rsqrtf(variance / (float)K + epsilon);
511     __syncthreads();
512     float4 reg_y;
513     reg_y.x = reg_x.x * s_variance * g;
514     reg_y.y = reg_x.y * s_variance * g;
515     reg_y.z = reg_x.z * s_variance * g;
516     reg_y.w = reg_x.w * s_variance * g;
517     if (idx < N * K)
518         FLOAT4(y[idx]) = reg_y;
519 }
520
521 // =====
522 // Phase 3c: Layer Normalization (2-pass reduce)
523 // =====
524 // 面试要点:
525 //   - Layer Norm:  $y = ((x - \text{mean}) / \text{std}) * g + b$ ,  $\text{std} = \sqrt{\text{variance}}$ ,  $\text{variance} = \text{mean}((x - \text{mean})^2)$ 
526 //   - 需要 2 次 block reduce: 先 mean (sum/K), 再 variance (sum((x-mean)^2)/K)
527 //   - 两次 __syncthreads 必须到位, 否则 s_mean 未对所有线程可见就计算 variance
528
529 template <const int NUM_THREADS = 128>
530 // Grid: (N, 1, 1), 一行一个 block
531 // Block: (128, 1, 1), NUM_THREADS=K=128
532 // source: LeetCode/kernels/layer-norm/layer_norm.cu
533 __global__ void layer_norm(float *x, float *y, float g, float b, int N, int K) {
534     int tid = threadIdx.x;
535     int bid = blockIdx.x;
536     int idx = bid * blockDim.x + threadIdx.x;
537     const float epsilon = 1e-5f;
538
539     __shared__ float s_mean;
540     __shared__ float s_variance;
541     float value = (idx < N * K) ? x[idx] : 0.0f;
542
543     // Pass 1: compute mean
544     float sum = block_reduce_sum<NUM_THREADS>(value);
545     if (tid == 0)
546         s_mean = sum / (float)K;
547     __syncthreads(); // 必须等待 s_mean 对所有线程可见
548
549     // Pass 2: compute variance = (x - mean)^2
550     float variance = (value - s_mean) * (value - s_mean);
551     variance = block_reduce_sum<NUM_THREADS>(variance);
552     if (tid == 0)
553         s_variance = rsqrtf(variance / (float)K + epsilon); // 1/std
554     __syncthreads(); // 必须等待 s_variance 对所有线程可见
555
556     if (idx < N * K)
557         y[idx] = ((value - s_mean) * s_variance) * g + b;
558 }
559
560 // Layer Norm + float4

```

```

561 template <const int NUM_THREADS = 128 / 4>
562 // Grid: (N, 1, 1)
563 // Block: (32, 1, 1), 128/4=32; 对应一行 K 元素按 4 个一组交给 32 个线程处理
564 // 注意: 该版本默认 K 按 4 对齐, 且输入/输出地址满足 float4 对齐
565 // source: LeetCode/kernels/layer-norm/layer_norm.cu
566 __global__ void layer_norm_vec4(float *x, float *y, float g, float b, int N,
567                                int K) {
568     int tid = threadIdx.x;
569     int bid = blockIdx.x;
570     int idx = (bid * blockDim.x + threadIdx.x) * 4;
571     const float epsilon = 1e-5f;
572
573     __shared__ float s_mean;
574     __shared__ float s_variance;
575     float4 reg_x = FLOAT4(x[idx]);
576     float value = (idx < N * K) ? (reg_x.x + reg_x.y + reg_x.z + reg_x.w) : 0.0f;
577
578     float sum = block_reduce_sum<NUM_THREADS>(value);
579     if (tid == 0)
580         s_mean = sum / (float)K;
581     __syncthreads();
582
583     float4 reg_x_hat;
584     reg_x_hat.x = reg_x.x - s_mean;
585     reg_x_hat.y = reg_x.y - s_mean;
586     reg_x_hat.z = reg_x.z - s_mean;
587     reg_x_hat.w = reg_x.w - s_mean;
588     float variance = reg_x_hat.x * reg_x_hat.x + reg_x_hat.y * reg_x_hat.y +
589                     reg_x_hat.z * reg_x_hat.z + reg_x_hat.w * reg_x_hat.w;
590     variance = block_reduce_sum<NUM_THREADS>(variance);
591     if (tid == 0)
592         s_variance = rsqrtf(variance / (float)K + epsilon);
593     __syncthreads();
594
595     float4 reg_y;
596     reg_y.x = reg_x_hat.x * s_variance * g + b;
597     reg_y.y = reg_x_hat.y * s_variance * g + b;
598     reg_y.z = reg_x_hat.z * s_variance * g + b;
599     reg_y.w = reg_x_hat.w * s_variance * g + b;
600     if (idx < N * K)
601         FLOAT4(y[idx]) = reg_y;
602 }
603
604 // =====
605 // Phase 4: GEMV — 矩阵向量乘 (M or N = 1, 纯 memory-bound 算子, warp-per-row 策略)
606 // =====
607 // 面试要点:
608 //   - GEMV 是典型的 memory-bound 算子:  $AI \approx O(1)$ , 瓶颈在内存带宽
609 //   - 核心策略: warp-per-row (每个 warp 负责矩阵的一行)
610 //   - 不同 K 值对应不同分块策略:
611 //       K=32 倍数: 一个 warp 的 32 线程恰好覆盖 K 维
612 //       K=128 倍数: 每个线程用 float4 处理 4 元素, warp 覆盖 128
613 //       K=16 < 32: 一个 warp 用不满, 用 ROW_PER_WARP=2 让每个 warp 处理 2 行
614 //
615 // a: MxK, x: Kx1, y: Mx1, 计算:  $y = a * x$ ; N = 1
616
617 // ---- SGEMV K32: 基础 warp-per-row ----
618 // 设计: block(32, 4), blockDim.x=WARP_SIZE=32 (K 需为 32 倍数时一轮覆盖, 否则内层循环 NUM_WARPS 次)
619 // grid(M/4), 每个 warp 负责一行
620 // K 为 32 的倍数时, warp 的 32 个线程恰好覆盖 K 维
621 // Grid: ((M + 3) / 4, 1, 1), 每 block 处理 4 行
622 // Block: (32, 4, 1), 每行 1 warp
623 // 注意: 该版本最适合 K 按 32 对齐; 当 K 更小时通常切到 K16 这类专用分支
624 // source: LeetCode/kernels/sgemv/sgemv.cu
625 __global__ void sgemv_k32(float *a, float *x, float *y, int M, int K) {
626     int tx = threadIdx.x; // 0~31
627     int ty = threadIdx.y; // 0~3
628     int bx = blockIdx.x;
629     int lane = tx % WARP_SIZE; // 0~31
630     int m = bx * blockDim.y + ty; // 全局行号

```

```

631 if (m < M) {
632     float sum = 0.0f;
633     // 沿 K 维的迭代数 = ceil(K/32), 每个 warp 要累加完整的K, 那么
634     // 每个thread就要负责累加NUM_ITERS个元素, NUM_ITERS = ceil(K/32)
635     const int NUM_ITERS = (K + WARP_SIZE - 1) / WARP_SIZE;
636 #pragma unroll
637     for (int w = 0; w < NUM_ITERS; ++w) {
638         // 假设K是32的整数倍, m * K 本行的起始地址, x: Kx1
639         int k = w * WARP_SIZE + lane;
640         sum += a[m * K + k] * x[k];
641     }
642     sum = warp_reduce_sum<WARP_SIZE>(sum);
643     // 每个 warp 处理一行, lane 0 写回结果
644     if (lane == 0)
645         y[m] = sum;
646 }
647 }
648
649 // ---- SGEMV K128: float4 向量化 ----
650 // 每个线程处理 4 个元素(float4), 一个 warp 覆盖 128 个元素
651 // Grid: ((M + 3) / 4, 1, 1)
652 // Block: (32, 4, 1)
653 // 注意: 该版本最适合 K 按 128 对齐, 且 x/a 的地址满足 float4 对齐
654 // source: LeetCode/kernels/sgemv/sgemv.cu
655 __global__ void sgemv_k128(float *a, float *x, float *y, int M, int K) {
656     int tx = threadIdx.x; // 0~31
657     int ty = threadIdx.y; // 0~3
658     int bx = blockIdx.x;
659     int lane = tx % WARP_SIZE; // 0~31
660     int m = blockDim.y * bx + ty;
661
662     if (m < M) {
663         float sum = 0.0f;
664         // 沿 K 维的迭代数 = ceil(K/128), 每个 warp 每轮用 float4 覆盖 128 个 K 元素
665         const int NUM_ITERS = (((K + WARP_SIZE - 1) / WARP_SIZE) + 4 - 1) / 4;
666 #pragma unroll
667         for (int w = 0; w < NUM_ITERS; ++w) {
668             int k = (w * WARP_SIZE + lane) * 4;
669             float4 reg_x = FLOAT4(x[k]);
670             float4 reg_a = FLOAT4(a[m * K + k]);
671             sum += (reg_a.x * reg_x.x + reg_a.y * reg_x.y + reg_a.z * reg_x.z +
672                 reg_a.w * reg_x.w);
673         }
674         sum = warp_reduce_sum<WARP_SIZE>(sum);
675         if (lane == 0)
676             y[m] = sum;
677     }
678 }
679
680 // ---- SGEMV K16: K < WarpSize, ROW_PER_WARP=2 ----
681 // 面试亮点: K=16 < 32, 一个 warp 可以处理多行
682 // ROW_PER_WARP=2, K_WARP_SIZE=16, 前 16 个 lane 处理 row0, 后 16 个 lane 处理 row1
683 template <const int ROW_PER_WARP = 2>
684 // Grid: ((M + 7) / 8, 1, 1), NUM_ROWS=8
685 // Block: (32, 4, 1)
686 // 注意: 这一版是面向 K=16 的专用写法; ROW_PER_WARP=2 时一个 warp 同时处理 2 行
687 // source: LeetCode/kernels/sgemv/sgemv.cu
688 __global__ void sgemv_k16(float *A, float *x, float *y, int M, int K) {
689     constexpr int K_WARP_SIZE = (WARP_SIZE + ROW_PER_WARP - 1) / ROW_PER_WARP; // 16
690     int tx = threadIdx.x;
691     int ty = threadIdx.y;
692     int bx = blockIdx.x;
693     int lane = tx % WARP_SIZE;
694     int k = lane % K_WARP_SIZE; // 0~15
695     int m = (blockDim.y * bx + ty) * ROW_PER_WARP + lane / K_WARP_SIZE;
696     if (m < M) {
697         float sum = A[m * K + k] * x[k];
698         // 按照K_WARP_SIZE=16, 分2组各自做 warp reduce sum, k==0的lane写回结果
699         sum = warp_reduce_sum<K_WARP_SIZE>(sum);
700         // 注意: 判断条件是 k == 0, 不是 lane == 0!

```

```

701     if (k == 0)
702         y[m] = sum;
703     }
704 }
705
706 // =====
707 // Phase 5: GEMM — 矩阵矩阵乘 (GPU 最重要的算子, 面试核心考点)
708 // =====
709 // 面试要点 (GEMM 优化五层金字塔):
710 //   Level 1 — Tiling (分块 + shared memory): 将数据从 HBM 搬到 SMEM 复用
711 //   Level 2 — Thread Tile (寄存器分块): 每个线程计算 TM×TN
712 //   个元素, 提高计算密度 Level 3 — Vectorize (向量化访存): float4/half2, 减少
713 //   load/store 指令数 Level 4 — Tensor Core (MMA
714 //   m16n8k16): 硬件矩阵乘单元, warp 级指令 Level 5 — Warp Specialization +
715 //   TMA (WGMMMA m64n128k16): Hopper 异步执行
716 //
717 // 计算密度递进:
718 //   Level 1: AI ≈ B_K / (2×sizeof) ≈ 32/8 = 4 → 仍是 memory-bound
719 //   Level 2: AI ≈ TM×TN×B_K / (2×sizeof) ≈ 8×8×8/8 = 64 → compute-bound
720 //   Level 4: Tensor Core 提供硬件加速的 256 FMA/cycle/warp → 大幅提升吞吐
721
722 // =====
723 // Phase 5a: SGEMM + HGEMM (非 Tensor Core 路径)
724 // =====
725
726 // ---- Level 1: SGEMM — Block Tile 32×32 + K Tile 32 ----
727 // 最基础的 tiling 实现, 演示 shared memory 的核心用法
728 // C = A x B, C[M, N] = A[M, K] x B[K, N]
729 // BM=BN=32, BK=32, block(32, 32), 一个线程计算 c 的一个元素
730 // Grid: ((N + 31) / 32, (M + 31) / 32, 1)
731 // Block: (32, 32, 1), 1024 线程
732 // source: LeetCUDA/kernels/sgemm/sgemm.cu
733 __global__ void sgemm(float *a, float *b, float *c, int M, int N, int K) {
734     constexpr int BM = 32; // vec 版: 32x4 = 128
735     constexpr int BN = 32; // vec 版: 32x4 = 128
736     constexpr int BK = 32;
737     __shared__ float s_a[BM][BK], s_b[BK][BN]; // 32x32x4=4KB smem, float = 4 bytes
738
739     int bx = blockIdx.x;
740     int by = blockIdx.y;
741     int tx = threadIdx.x;
742     int ty = threadIdx.y;
743     int tid = threadIdx.y * blockDim.x + tx;
744
745     // 线程到 smem 的映射: 32x32 线程, 每个线程加载 a 和 b 各 1 个元素
746     int load_smem_a_m = tid / 32; // row 0~31 由 32 线程加载; vec 版: a_m = tid / (32 / 4), row 0~127, [128x32]
747     int load_smem_a_k = tid % 32; // col 0~31 由 32 线程加载; vec 版: a_k = (tid % (32 / 4)) * 4, t 0~7, col 0~31, 每个线程加载 4
748     // 个元素, 8x4 = 32
749     int load_smem_b_k = tid / 32; // row 0~31 由 32 线程加载; vec 版: b_k = tid / 32, row 0~31, [32x128]
750     int load_smem_b_n = tid % 32; // col 0~31 由 32 线程加载; vec 版: b_n = (tid % 32) * 4, t 0~31, col 0~127, 每个线程加载 4 个元
751     // 素, 32x4 = 128
752     int load_gmem_a_m = by * BM + load_smem_a_m; // gmem row; vec 版: load_smem_a_m, 0~127, 0, 1, 2, ... (连续) [128x128]
753     int load_gmem_b_n = bx * BN + load_smem_b_n; // gmem col; vec 版: load_smem_b_n, 0~127, 0, 4, 8, ... (间隔)
754
755     float sum = 0.f; // 遍历完整的K, slice K; vec 版: sum[4][4] = 0.f; 每个线程处理4x4的大小, 才能保证32x32线程处理[32x4, 32x4]=[128
756     // x128]大小
757     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
758         int load_gmem_a_k = bk * BK + load_smem_a_k; // A [M, K]
759         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
760         // vec 版: FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr])
761         s_a[load_smem_a_m][load_smem_a_k] = a[load_gmem_a_addr];
762         int load_gmem_b_k = bk * BK + load_smem_b_k; // B [K, N]
763         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
764         // vec 版: FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);
765         s_b[load_smem_b_k][load_smem_b_n] = b[load_gmem_b_addr];
766         __syncthreads(); // 确保整个 smem tile 加载完毕
767
768 #pragma unroll
769         for (int k = 0; k < BK; ++k) {
770             int comp_smem_a_m = load_smem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续)
771             int comp_smem_b_n = load_smem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
772             sum += s_a[comp_smem_a_m][k] * s_b[k][comp_smem_b_n];
773         }
774     }
775 }

```

```

771     __syncthreads(); // 确保 smem 不会在下一轮加载时被覆盖
772 }
773 int store_gmem_c_m = load_gmem_a_m; // vec 版: 0~127, 0, 1, 2, ... (连续) [128x128]
774 int store_gmem_c_n = load_gmem_b_n; // vec 版: 0~127, 0, 4, 8, ... (间隔)
775 int store_gmem_c_addr = store_gmem_c_m * N + store_gmem_c_n;
776 c[store_gmem_c_addr] = sum; // C [M, N] = A[M, K] x B[K, N]
777 }
778
779 // ---- Level 1: SGEMM Vec4 — Block Tile 128x128 + K Tile 32 + Thread Tile 4x4 ----
780 // 在 Level 1 基础上引入两层优化:
781 // 1) float4 向量化加载: A/B 各用 1 条 128-bit load 取代 4 条 32-bit load
782 // 2) Thread Tile 4x4: 每线程计算 16 个 C 元素, 提升计算/访存比 (AI 从
783 //    BK/2≈16 提升到 TM*TN*BK/2≈256), 减少线程总数带来的同步开销
784 // C = A x B, C[M, N] = A[M, K] x B[K, N], A/B 均 row-major
785 // BM=BN=128, BK=32, block(32, 32)=1024 线程, 每线程负责 4x4=16 个 C 元素
786 // 1024 x 16 = 16384 = 128 x 128 ✓
787 //
788 // 线程到 4x4 tile 的映射 (与加载映射解耦, 独立计算更清晰):
789 // m_tile = tid / 32 (0~31), 每 tile 4 行 → 行 [m_tile*4, m_tile*4+3], 覆盖 0~127
790 // n_tile = tid % 32 (0~31), 每 tile 4 列 → 列 [n_tile*4, n_tile*4+3], 覆盖 0~127
791 //
792 // 加载映射 (每线程 4 个元素, float4):
793 // A[128][32]: a_m = tid/8 (8 线程/行), a_k = (tid%8)*4 (4 列/线程) → 8x4=32 列 ✓
794 // B[32][128]: b_k = tid/32 (32 线程/行), b_n = (tid%32)*4 (4 列/线程) → 32x4=128 列 ✓
795 // row-major 下 A[m][k..k+3] 与 B[k][n..n+3] 均连续 → float4 load 合法
796 //
797 // △ Bank Conflict 提示 (面试加分点):
798 // s_b[32][128] 上 warp 内 32 线程按 stride=4 访问 (tid%32 决定列 0,4,8,...,124)
799 // → 每 4 个线程落同一 bank 不同地址 → 4-way bank conflict。生产代码可用
800 // s_b[BK][BN+1] PAD 打散, 这里保持最简布局便于讲解。
801 //
802 // Grid: ((N + 127) / 128, (M + 127) / 128, 1)
803 // Block: (32, 32, 1), 1024 线程
804 // 假设: M/N 为 128 的倍数, K 为 32 的倍数 (与 Level 1 naive 版一致的边界约定)
805 // source: LeetCode/kernels/sgemm/sgemm.cu (vec4 variant)
806 __global__ void sgemm_vec4(float *a, float *b, float *c, int M, int N, int K) {
807     constexpr int BM = 128;
808     constexpr int BN = 128;
809     constexpr int BK = 32;
810     __shared__ float s_a[BM][BK]; // 128*32*4 = 16KB, float = 4 bytes
811     __shared__ float s_b[BK][BN]; // 32*128*4 = 16KB
812
813     int bx = blockIdx.x;
814     int by = blockIdx.y;
815     int tx = threadIdx.x;
816     int ty = threadIdx.y;
817     int tid = threadIdx.y * blockDim.x + tx; // 0~1023
818
819     // 加载 A: 每线程加载 s_a[a_m][a_k..a_k+3] 共 4 个元素
820     int load_smem_a_m = tid / 8; // 0~127, 8 线程/行
821     int load_smem_a_k = (tid % 8) * 4; // 0,4,...,28
822     // 加载 B: 每线程加载 s_b[b_k][b_n..b_n+3] 共 4 个元素
823     int load_smem_b_k = tid / 32; // 0~31, 32 线程/行
824     int load_smem_b_n = (tid % 32) * 4; // 0,4,...,124
825
826     int load_gmem_a_m = by * BM + load_smem_a_m;
827     int load_gmem_b_n = bx * BN + load_smem_b_n;
828
829     // 4x4 Thread Tile 基址 (独立于加载映射, 避免 /4 漏乘 bug)
830     int comp_smem_a_m_base = (tid / 32) * 4; // 0,4,8,...,124
831     int comp_smem_b_n_base = (tid % 32) * 4; // 0,4,8,...,124
832
833     float sum[4][4] = {0.f};
834     for (int bk = 0; bk < (K + BK - 1) / BK; ++bk) {
835         int load_gmem_a_k = bk * BK + load_smem_a_k;
836         int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k;
837         FLOAT4(s_a[load_smem_a_m][load_smem_a_k]) = FLOAT4(a[load_gmem_a_addr]);
838         int load_gmem_b_k = bk * BK + load_smem_b_k;
839         int load_gmem_b_addr = load_gmem_b_k * N + load_gmem_b_n;
840         FLOAT4(s_b[load_smem_b_k][load_smem_b_n]) = FLOAT4(b[load_gmem_b_addr]);

```

```

841     __syncthreads();
842
843 #pragma unroll
844 for (int k = 0; k < BK; ++k) {
845     // 每次迭代加载 4 个 A 元素 + 4 个 B 元素, 再做 4x4=16 次 FMA
846     float a_vals[4] = {s_a[comp_smem_a_m_base + 0][k],
847                        s_a[comp_smem_a_m_base + 1][k],
848                        s_a[comp_smem_a_m_base + 2][k],
849                        s_a[comp_smem_a_m_base + 3][k]};
850     float b_vals[4] = {s_b[k][comp_smem_b_n_base + 0],
851                        s_b[k][comp_smem_b_n_base + 1],
852                        s_b[k][comp_smem_b_n_base + 2],
853                        s_b[k][comp_smem_b_n_base + 3]};
854 #pragma unroll
855     for (int i = 0; i < 4; ++i) {
856 #pragma unroll
857         for (int j = 0; j < 4; ++j) {
858             sum[i][j] += a_vals[i] * b_vals[j];
859         }
860     }
861 }
862 __syncthreads();
863 }
864
865 // 存储 4x4: 每行 4 个元素连续 → 可用 float4 store (要求 N 为 4 的倍数以保证对齐)
866 int store_gmem_c_m = by * BM + comp_smem_a_m_base;
867 int store_gmem_c_n = bx * BN + comp_smem_b_n_base;
868 #pragma unroll
869 for (int i = 0; i < 4; ++i) {
870     int store_gmem_c_addr = (store_gmem_c_m + i) * N + store_gmem_c_n;
871     float4 reg_c;
872     reg_c.x = sum[i][0];
873     reg_c.y = sum[i][1];
874     reg_c.z = sum[i][2];
875     reg_c.w = sum[i][3];
876     FLOAT4(c[store_gmem_c_addr]) = reg_c;
877 }
878 }
879
880 // =====
881 // Phase 5b: HGEMM — Tensor Core 路径 (MMA m16n8k16 + WGMMMA m64n128k16)
882 // =====
883 // 面试要点:
884 //   - MMA (Matrix Multiply-Accumulate): Ampere+ 的 Tensor Core 指令
885 //   - m16n8k16 含义: M=16, N=8, K=16 的矩阵乘, 结果 [16x8] = [16x16]x[16x8]
886 //   - ldmatrix: 从 shared memory 加载 16x16 的矩阵片段到寄存器 (4 条 32-bit
887 //     寄存器)
888 //   - Multistage Pipeline: s2/s3/s4 个 stage, 用 cp.async 异步加载下一批数据
889 //   - Block Swizzle: 在 grid 维度做 swizzle, 改善 L2 cache locality
890 //
891 // ★ TN 布局详解 (面试高频考点):
892 //   TN 命名约定: 来自 BLAS 的 op(A) × op(B) 语义
893 //   BLAS 源自 Fortran, 默认列优先 (column-major) 存储
894 //   N = Normal (列优先, BLAS 原生格式)
895 //   T = Transposed (行优先, 相对 BLAS 来说是"转置过的")
896 //   第一个字母 → A 的 op, 第二个字母 → B 的 op
897 //   所以 TN 表示: A 是行优先 (相对 BLAS=Transposed), B 是列优先 (相对
898 //     BLAS=Normal) 即: C = op(A) × op(B) = A^T × B? 不对! 在 row-major
899 //     视角下: TN = A row-major [MxK], B^T row-major [NxK] (等价于 B col-major [KxN]) 在 cuBLAS
900 //     调用中: cublasGemmEx(..., CUBLAS_OP_T, CUBLAS_OP_N, ...)
901 //     T on A: BLAS 把 row-major 的 A 视为 A^T, 传 T 表示"转置回去"
902 //     N on B: B 已经是 BLAS 原生的 col-major, 无需转置
903 //
904 //   记忆口诀: TN = A行(T) B列(N), 第一字母 A 第二字母 B
905 //   T = row-major (行优先, 对 BLAS 来说是 transposed)
906 //   N = col-major (列优先, BLAS native = normal)
907 //
908 //   LeetCUDA _nn 布局对比 (A/B 均 row-major 自然存储): C[MxN] = A[MxK] × B[KxN]
909 //   - A: row-major [M, K], B: row-major [K, N]
910 //   - 按 N=col-major/T=row-major 约定, 二者 BLAS 视角均为 T → cuBLAS 等效 (T,T)

```

```

911 // - 问题: ldmatrix 默认加载 col-major, B 是 row-major 需要 .trans
912 //
913 // TN 布局:  $C[M \times N] = A[M \times K] \times B[K \times N]$ , B 以  $B^T = [N \times K]$  row-major 存储 (A 行优先, B 列优先)
914 // - A  $[M \times K]$ : row-major  $\rightarrow$  全局索引  $A[m \times K + k]$ , smem s_a[BM][BK]
915 // -  $B^T [N \times K]$ : row-major  $\rightarrow$  全局索引  $B[n \times K + k]$  即访问原 B 元素 (k,n) ( $\Delta$  内维连续的是 K)
916 // - 优势:  $B^T$  已是 row-major, ldmatrix 无需 .trans, 天然匹配 MMA row.col
917 // MMA 指令: mma.sync.aligned.m16n8k16.row.col
918 // - row.col: A 输入 row-major, B 输入 col-major  $\rightarrow$  与 TN 布局天然匹配
919 //
920 // WGMMA (Warp Group MMA, Hopper+):
921 // - warpgroup 级指令 (128 threads = 4 warps), 异步执行
922 // - m64n128k16: 一次处理  $64 \times 128 \times 16$  的矩阵乘 (总 tile 量 131072, 是 MMA m16n8k16 的 64 倍)
923 // - Warp Specialization: Producer(128 threads) 做 TMA 搬运, Consumer(128
924 // threads) 做计算
925 // - TMA: 硬件 DMA, ~零寄存器开销, 支持 1D~5D 寻址
926
927 // =====
928 // Phase 5b-1: MMA PTX 宏定义
929 // =====
930
931 // ---- gmem  $\rightarrow$  smem: cp.async ----
932 #define CP_ASYNC_COMMIT_GROUP() asm volatile("cp.async.commit_group;\n" ::)
933 #define CP_ASYNC_WAIT_ALL() asm volatile("cp.async.wait_all;\n" ::)
934 #define CP_ASYNC_WAIT_GROUP(n) \
935     asm volatile("cp.async.wait_group %0;\n" :: "n"(n))
936
937 // cp.async.cg: bypass L1, 写入 L2 (适合 GEMM 中只读一次的数据)
938 // cp.async.ca: cache all, L1+L2 (适合需要多次复用的数据)
939 // 注意: cg 只支持 16 bytes, ca 支持 4/8/16 bytes
940 #define CP_ASYNC_CG(dst, src, bytes) \
941     asm volatile( \
942         "cp.async.cg.shared.global.L2::128B [%0], [%1], %2;\n" :: "r"(dst), \
943         "l"(src), "n"(bytes))
944
945 // ---- ldmatrix: smem  $\rightarrow$  register (Tensor Core 专用) ----
946 // ldmatrix.sync.aligned.xN.m8n8.shared.b16
947 // 每次加载  $8 \times 8$  的 half 矩阵片段到 1/2/4 条 32-bit 寄存器
948 // aligned: 要求 128-bit 对齐
949 // trans: 转置加载 (用于 col-major 的 B 矩阵)
950 #define LDMATRIX_X4(R0, R1, R2, R3, addr) \
951     asm volatile( \
952         "ldmatrix.sync.aligned.x4.m8n8.shared.b16 {%0, %1, %2, %3}, [%4];\n" \
953         : "=r"(R0), "=r"(R1), "=r"(R2), "=r"(R3) \
954         : "r"(addr))
955
956 #define LDMATRIX_X2(R0, R1, addr) \
957     asm volatile("ldmatrix.sync.aligned.x2.m8n8.shared.b16 {%0, %1}, [%2];\n" \
958         : "=r"(R0), "=r"(R1) \
959         : "r"(addr))
960
961 // ldmatrix.x2.trans: 转置加载 (用于 NN 布局中需要 col-major B 矩阵的场景)
962 // FA 中 V[Bc,d] 为 row-major, 但 P@V 的 MMA 需要 col-major 的 B  $\rightarrow$  使用 trans
963 // 加载
964 #define LDMATRIX_X2_T(R0, R1, addr) \
965     asm volatile( \
966         "ldmatrix.sync.aligned.x2.trans.m8n8.shared.b16 {%0, %1}, [%2];\n" \
967         : "=r"(R0), "=r"(R1) \
968         : "r"(addr))
969
970 // ---- mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 ----
971 // m16n8k16: M=16, N=8, K=16 (Ampere Tensor Core 的基本 tile)
972 // row.col: A 是 row-major, B 是 col-major
973 // f16.f16.f16.f16: A/B 是 f16, C/D 是 f16 (f32 累加版本用 f32.f16.f16.f32)
974 // 2 个输出寄存器 (RD0, RD1), 4 个 A 寄存器 + 2 个 B 寄存器
975 // C 矩阵大小  $16 \times 8 = 128$  元素 = 128 个 half; 32 线程分担, 每线程 4 half = 2 个 uint32
976 #define HMMA16816(RD0, RD1, RA0, RA1, RA2, RA3, RB0, RB1, RC0, RC1) \
977     asm volatile( \
978         "mma.sync.aligned.m16n8k16.row.col.f16.f16.f16.f16 {%0, %1}, {%2, %3, " \
979         "%4, %5}, {%6, %7}, {%8, %9};\n" \
980         : "=r"(RD0), "=r"(RD1) \

```



```

981 : "r"(RA0), "r"(RA1), "r"(RA2), "r"(RA3), "r"(RB0), "r"(RB1), "r"(RC0), \
982 "r"(RC1))
983
984 // ---- MMA 辅助函数 ----
985 // div_ceil: 整数除法向上取整
986 #define HOST_DEVICE_INLINE __device__ __host__ inline
987 HOST_DEVICE_INLINE int div_ceil(int a, int b) {
988     return (a % b != 0) ? (a / b + 1) : (a / b);
989 }
990
991 // =====
992 // Phase 5b-2: HGMEM MMA — m16n8k16 + multistage pipeline + TN 布局
993 // =====
994 // 面试重点 — Tile Hierarchy:
995 //   MMA Atom:          m16n8k16 (1 条 MMA 指令处理的最小 tile)
996 //   MMA Tile (warp):    2x4=8 个 MMA atom = [32x32] (一个 warp 的计算)
997 //   Warp Tile:         4x4=16 warps = [128x128] (整个 block 的计算)
998 //   线程映射:          2x4=8 warps, 每个 warp 4x1=4 个 warp tile
999 // 实际: MMA_TILE_M=2, MMA_TILE_N=4, WARP_TILE_M=4, WARP_TILE_N=4
1000 //   → BM=16x2x4=128, BN=8x4x4=128, Warps=2x4=8, Threads=8x32=256
1001 //
1002 // TN 布局在本 kernel 中的体现 (T=A 行优先, N=B 列优先):
1003 //   - A[M][K]: row-major, 全局索引 A[m*K + k], shared memory s_a[BM][BK]
1004 //   - B[K][N]: col-major (等价于 B^T[N][K] row-major), 全局索引 B[n*K+k] 即原 B 元素 (k,n)
1005 //     (△ 内维连续的是 K) shared memory s_b[BN][BK] = s_b[N_tile][K_tile]
1006 //   - ldmatrix A: 用 x4 (非转置), 因为 A 是 row-major, ldmatrix 原生匹配
1007 //   - ldmatrix B: 用 x2 (非转置), 因为 B^T 在 smem 中为 row-major, ldmatrix 逐行加载
1008 //     B^T 的行即 B 的列, 天然匹配 MMA row.col 的 col-major B 输入, 无需 .trans
1009 //   - MMA 指令: row.col = A row, B col → 天然匹配 TN 布局
1010 // Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1011 //   - grid.z = S 个 swizzle 分区
1012 //   - grid.x = 每个分区内部需要发射多少个 128x128 的 N tiles
1013 //   - bx = blockIdx.z * gridDim.x + blockIdx.x, 把连续 block 打散到不同 N 区域, 改善 L2 命中
1014 // Block: (256, 1, 1), 8 warps
1015 // source: LeetCUDA/kernels/hgemm/mma/basic/hgemm_mma_stage_tn.cu
1016 template <const int MMA_M = 16, const int MMA_N = 8, const int MMA_K = 16,
1017           const int MMA_TILE_M = 2, const int MMA_TILE_N = 4,
1018           const int WARP_TILE_M = 4, const int WARP_TILE_N = 4,
1019           const int K_STAGE = 3, const bool BLOCK_SWIZZLE = false>
1020 __global__ void __launch_bounds__(256)
1021     hgemm_mma_stages_tn(half *A, half *B, half *C, int M, int N, int K) {
1022     // Block Swizzle: 在 grid x 维度做 swizzle, 改善 L2 cache 局部性
1023     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1024     const int by = blockIdx.y;
1025     constexpr int BM = MMA_M * MMA_TILE_M * WARP_TILE_M; // 16*2*4=128
1026     constexpr int BN = MMA_N * MMA_TILE_N * WARP_TILE_N; // 8*4*4=128
1027     constexpr int BK = MMA_K; // 16
1028
1029     // Dynamic shared memory: K_STAGE 个 stage 的 A 和 B
1030     // TN 布局: s_a[BM][BK]=[128][16] (A row-major), s_b[BN][BK]=[128][16] (B^T
1031     // row-major, 即 B col-major [KxN] 在 smem 中按 B^T[NxK] 存储)
1032     // 原始实现会按配置决定是否给 A/B 的 K 维加 PAD; 尤其 B 在 TN 布局下常见会额外
1033     // 加 B_PAD 来打散 bank 映射, 避免按列访问时出现明显 bank conflict. 这里先保留最简 PAD=0 版本。
1034     extern __shared__ half smem[];
1035     half *s_a = smem;
1036     half *s_b = smem + K_STAGE * BM * BK; // A 和 B 连续存放
1037     constexpr int s_a_stage_offset = BM * BK; // 128*16
1038     constexpr int s_b_stage_offset =
1039         BN * BK; // 128*16 △ BN(128)xBK(16) = B^T row-major 的 smem 布局
1040
1041     const int tid = threadIdx.y * blockDim.x + threadIdx.x;
1042     const int warp_id = tid / WARP_SIZE; // 0~7
1043     const int lane_id = tid % WARP_SIZE; // 0~31
1044     const int warp_m = warp_id % 2; // 0,1 (M 方向 2 个 warp)
1045     const int warp_n = warp_id / 2; // 0,1,2,3 (N 方向 4 个 warp)
1046
1047     // 线程到 global memory 的映射 (用于加载 A 和 B)
1048     // TN 布局关键: A[m*K+k] 是 row-major, B^T[n*K+k] 是 row-major (内维连续的是 K)
1049     int load_smem_a_m = tid / 2; // 0~127
1050     int load_smem_a_k = (tid % 2 == 0) ? 0 : 8; // 0, 8

```



```

1051 int load_smem_b_n = tid / 2; // 0~127 → B^T 的 N 方向 (row-major 的行)
1052 int load_smem_b_k =
1053     (tid % 2 == 0) ? 0 : 8; // 0, 8 → B^T 的 K 方向 (row-major 的列)
1054 int load_gmem_a_m = by * BM + load_smem_a_m;
1055 int load_gmem_b_n =
1056     bx * BN + load_smem_b_n; // B 全局列号 = N 方向的 tile 起始 + 线程偏移
1057 if (load_gmem_a_m >= M || load_gmem_b_n >= N)
1058     return;
1059
1060 // 累加器: 每个 warp 计算 WARP_TILE_M×WARP_TILE_N=16 个 MMA tile
1061 uint32_t RC[WARP_TILE_M][WARP_TILE_N][2] = {}; // 初始化为 0
1062
1063 // CVTA: 一次转换 smem 基地址, 避免每次 cp.async 都做转换
1064 uint32_t smem_a_base_ptr = __cvta_generic_to_shared(s_a);
1065 uint32_t smem_b_base_ptr = __cvta_generic_to_shared(s_b);
1066
1067 // 预加载前 (K_STAGE-1) 个 stage
1068 // TN 布局: A 的 gmem 索引用 m*K+k(row-major), B^T 用 n*K+k(row-major, 即 B col-major [K×N])
1069 #pragma unroll
1070 for (int k = 0; k < (K_STAGE - 1); ++k) {
1071     int load_gmem_a_k = k * BK + load_smem_a_k;
1072     int load_gmem_a_addr = load_gmem_a_m * K + load_gmem_a_k; // A: [m][k]
1073     int load_gmem_b_k = k * BK + load_smem_b_k;
1074     int load_gmem_b_addr =
1075         load_gmem_b_n * K + load_gmem_b_k; // B^T: [n][k] row-major (即 B[k][n] col-major) △
1076
1077     uint32_t load_smem_a_ptr =
1078         (smem_a_base_ptr +
1079          (k * s_a_stage_offset + load_smem_a_m * BK + load_smem_a_k) *
1080           sizeof(half));
1081     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);
1082
1083     uint32_t load_smem_b_ptr =
1084         (smem_b_base_ptr +
1085          (k * s_b_stage_offset + load_smem_b_n * BK + load_smem_b_k) *
1086           sizeof(half));
1087     CP_ASYNC_CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1088
1089     CP_ASYNC_COMMIT_GROUP();
1090 }
1091
1092 const int NUM_K_TILES = div_ceil(K, BK);
1093 CP_ASYNC_WAIT_GROUP(K_STAGE - 2); // 等待前 (K_STAGE-2) 个 group 完成
1094 __syncthreads();
1095
1096 // 主循环: K 维分块迭代
1097 #pragma unroll
1098 for (int k = (K_STAGE - 1); k < NUM_K_TILES; ++k) {
1099     // Stage 选择: 轮转方式 (round-robin)
1100     // 这里统一用 %K_STAGE, 而不是只在 s2/s4 时写成 &1 / &3:
1101     // 原始实现要兼容 s3 这类非 2 的幂 stage 数, 因此直接用 mod 最稳妥。
1102     int smem_sel = (k + 1) % K_STAGE; // 当前计算的 stage
1103     int smem_sel_next = k % K_STAGE; // 下一轮加载的 stage
1104
1105     // 异步加载下一批数据到 smem_sel_next
1106     // TN 布局: A 的 gmem 地址用 m*K+k (row-major), B^T 的 gmem 地址用
1107     // n*K+k (row-major, 内维连续的是 K)
1108     int load_gmem_a_k = k * BK + load_smem_a_k;
1109     int load_gmem_a_addr =
1110         load_gmem_a_m * K + load_gmem_a_k; // A: row-major [m][k]
1111     int load_gmem_b_k = k * BK + load_smem_b_k;
1112     int load_gmem_b_addr =
1113         load_gmem_b_n * K + load_gmem_b_k; // B^T: row-major [n][k], 内维连续的是 K △
1114
1115     uint32_t load_smem_a_ptr =
1116         (smem_a_base_ptr + (smem_sel_next * s_a_stage_offset +
1117          load_smem_a_m * BK + load_smem_a_k) *
1118          sizeof(half));
1119     CP_ASYNC_CG(load_smem_a_ptr, &A[load_gmem_a_addr], 16);

```

```

1121 uint32_t load_smem_b_ptr =
1122     (smem_b_base_ptr + (smem_sel_next * s_b_stage_offset +
1123         load_smem_b_n * BK + load_smem_b_k) *
1124         sizeof(half));
1125 CP_ASYNC.CG(load_smem_b_ptr, &B[load_gmem_b_addr], 16);
1126 CP_ASYNC.COMMIT_GROUP();
1127
1128 // ldmatrix: 从 smem_sel 加载 A 和 B 到寄存器
1129 // TN 布局关键: A 用 x4 (非转置), 因为 A 是 row-major
1130 //           B 用 x2 (非转置), smem 中 B^T 为 row-major, 逐行加载即得 B 的列, 天然匹配 col-major B
1131 uint32_t RA[WARP_TILE_M][4];
1132 uint32_t RB[WARP_TILE_N][2];
1133
1134 // ldmatrix.x4: 加载 A 的 m16k16 片段 (row-major A, 非转置)
1135 #pragma unroll
1136 for (int i = 0; i < WARP_TILE_M; ++i) {
1137     int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1138     int lane_smem_a_m =
1139         warp_smem_a_m + lane_id % 16; // ldmatrix 用 16 个 lane
1140     int lane_smem_a_k = (lane_id / 16) * 8; // 0, 8
1141     uint32_t lane_smem_a_ptr =
1142         (smem_a_base_ptr +
1143         (smem_sel * s_a_stage_offset + lane_smem_a_m * BK + lane_smem_a_k) *
1144         sizeof(half));
1145     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1146 }
1147
1148 // ldmatrix.x2: 加载 B 的 k16n8 片段 (非转置)
1149 // 为什么不用 .trans? 因为 smem 中存的是 B^T row-major [N][K],
1150 // ldmatrix 逐行加载 B^T 的行 = B 的列, 天然给出 col-major B fragment → 直接匹配 MMA row.col
1151 #pragma unroll
1152 for (int j = 0; j < WARP_TILE_N; ++j) {
1153     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1154     int lane_smem_b_n =
1155         warp_smem_b_n + lane_id % 8; // ldmatrix B 用 8 个 lane
1156     int lane_smem_b_k = ((lane_id / 8) % 2) * 8; // 0, 8
1157     uint32_t lane_smem_b_ptr =
1158         (smem_b_base_ptr +
1159         (smem_sel * s_b_stage_offset + lane_smem_b_n * BK + lane_smem_b_k) *
1160         sizeof(half));
1161     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1162 }
1163
1164 // MMA compute: 发射 WARP_TILE_M × WARP_TILE_N 条 MMA 指令
1165 #pragma unroll
1166 for (int i = 0; i < WARP_TILE_M; ++i) {
1167     #pragma unroll
1168     for (int j = 0; j < WARP_TILE_N; ++j) {
1169         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1170             RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1171     }
1172 }
1173
1174 // 等待当前 stage 的异步加载完成, 然后 sync
1175 CP_ASYNC.WAIT_GROUP(K_STAGE - 2);
1176 __syncthreads();
1177 }
1178
1179 // 尾端处理: 最后 (K_STAGE-1) 个 stage 的计算 (无新数据加载)
1180 if ((K_STAGE - 2) > 0) {
1181     CP_ASYNC.WAIT_GROUP(0);
1182     __syncthreads();
1183 }
1184
1185 {
1186 #pragma unroll
1187 for (int k = 0; k < (K_STAGE - 1); k++) {
1188     uint32_t RA[WARP_TILE_M][4];
1189     uint32_t RB[WARP_TILE_N][2];
1190     int stage_sel = ((NUM_K_TILES - (K_STAGE - 1) + k) % K_STAGE);

```

```

1191
1192 #pragma unroll
1193 for (int i = 0; i < WARP_TILE_M; ++i) {
1194     int warp_smem_a_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1195     int lane_smem_a_m = warp_smem_a_m + lane_id % 16;
1196     int lane_smem_a_k = (lane_id / 16) * 8;
1197     uint32_t lane_smem_a_ptr =
1198         (smem_a_base_ptr + (stage_sel * s_a_stage_offset +
1199             lane_smem_a_m * BK + lane_smem_a_k) *
1200             sizeof(half));
1201     LDMATRIX_X4(RA[i][0], RA[i][1], RA[i][2], RA[i][3], lane_smem_a_ptr);
1202 }
1203 #pragma unroll
1204 for (int j = 0; j < WARP_TILE_N; ++j) {
1205     int warp_smem_b_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1206     int lane_smem_b_n = warp_smem_b_n + lane_id % 8;
1207     int lane_smem_b_k = ((lane_id / 8) % 2) * 8;
1208     uint32_t lane_smem_b_ptr =
1209         (smem_b_base_ptr + (stage_sel * s_b_stage_offset +
1210             lane_smem_b_n * BK + lane_smem_b_k) *
1211             sizeof(half));
1212     LDMATRIX_X2(RB[j][0], RB[j][1], lane_smem_b_ptr);
1213 }
1214 #pragma unroll
1215 for (int i = 0; i < WARP_TILE_M; ++i) {
1216     #pragma unroll
1217     for (int j = 0; j < WARP_TILE_N; ++j) {
1218         HMMA16816(RC[i][j][0], RC[i][j][1], RA[i][0], RA[i][1], RA[i][2],
1219             RA[i][3], RB[j][0], RB[j][1], RC[i][j][0], RC[i][j][1]);
1220     }
1221 }
1222 }
1223 }
1224
1225 // Epilogue: 寄存器 → global memory (通过 warp shuffle + 128-bit store)
1226 {
1227     for (int i = 0; i < WARP_TILE_M; ++i) {
1228         uint32_t RC0[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1229         uint32_t RC1[WARP_TILE_N][4]; // 32 bits x 4 = 128 bits = 8 half
1230         // 用 warp shuffle 收集同一个 MMA tile 内不同 lane 的结果。
1231         // 对 m16n8k16 的 C fragment:
1232         //   - lane 0/4/8/.../28 各自持有某一行里的 {c0,c1}
1233         //   - lane+1 持有同一行里的 {c2,c3}
1234         //   - lane+2 持有同一行里的 {c4,c5}
1235         //   - lane+3 持有同一行里的 {c6,c7}
1236         // 所以每个 4-lane 子组可以通过 shfl 收齐一整行 8 个 half, 然后由 lane%4==0
1237         // 的那个线程做一次 128-bit store。
1238     #pragma unroll
1239     for (int j = 0; j < WARP_TILE_N; ++j) {
1240         RC0[j][0] = RC[i][j][0];
1241         RC1[j][0] = RC[i][j][1];
1242         RC0[j][1] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 1);
1243         RC0[j][2] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 2);
1244         RC0[j][3] = __shfl_sync(0xffffffff, RC[i][j][0], lane_id + 3);
1245         RC1[j][1] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 1);
1246         RC1[j][2] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 2);
1247         RC1[j][3] = __shfl_sync(0xffffffff, RC[i][j][1], lane_id + 3);
1248     }
1249     // 每 4 个 lane 中只有 lane 0 做 128-bit store
1250     if (lane_id % 4 == 0) {
1251         int store_warp_smem_c_m = warp_m * (MMA_M * WARP_TILE_M) + i * MMA_M;
1252         int store_lane_gmem_c_m = by * BM + store_warp_smem_c_m + lane_id / 4;
1253     #pragma unroll
1254     for (int j = 0; j < WARP_TILE_N; ++j) {
1255         int store_warp_smem_c_n = warp_n * (MMA_N * WARP_TILE_N) + j * MMA_N;
1256         int store_lane_gmem_c_n = bx * BN + store_warp_smem_c_n;
1257         int store_gmem_c_addr_0 =
1258             store_lane_gmem_c_m * N + store_lane_gmem_c_n;
1259         int store_gmem_c_addr_1 =
1260             (store_lane_gmem_c_m + 8) * N + store_lane_gmem_c_n;

```

```

1261 // 128-bit store: 一次写入 8 个 half
1262 *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_0]) =
1263     *reinterpret_cast<float4 *>(&RC0[j][0]);
1264 *reinterpret_cast<float4 *>(&C[store_gmem_c_addr_1]) =
1265     *reinterpret_cast<float4 *>(&RC1[j][0]);
1266 }
1267 }
1268 }
1269 }
1270 }
1271
1272 // =====
1273 // Phase 5b-3: HGEMM WGMMMA — m64n128k16 + TMA + Warp Specialization (Hopper)
1274 // =====
1275 // 面试要点 (WGMMMA vs MMA 对比):
1276 //   - MMA: warp 级 (32 threads), 同步执行
1277 //   - WGMMMA: warpgroup 级 (128 threads = 4 warps), 异步执行 (fire-and-forget)
1278 //   - m64n128k16: M=64, N=128, K=16 → 一次处理 64×128×16=131K 个乘加 (MMA 的 32
1279 //     倍)
1280 //   - TMA (Tensor Memory Accelerator): 硬件 DMA, 2D 寻址, 零寄存器开销
1281 //   - Warp Specialization: Producer 做 TMA, Consumer 做 WGMMMA, 通过 barrier
1282 //     同步
1283 //   - 128B swizzle: shared memory 的 128B swizzle 模式, 避免 bank conflict
1284
1285 // ---- WGMMMA 辅助函数 ----
1286 #define WGMMMA_FENCE() asm volatile("wgmma.fence.sync.aligned;\n" ::: "memory")
1287 #define WGMMMA_COMMIT_GROUP() \
1288     asm volatile("wgmma.commit_group.sync.aligned;\n" ::: "memory")
1289 #define WGMMMA_WAIT_GROUP(n) \
1290     asm volatile("wgmma.wait_group.sync.aligned %0;\n" :::"n"(n) : "memory")
1291
1292 // Shared memory descriptor encode: 将 smem 地址编码为 WGMMMA 可用的描述符
1293 #define SMEM_DESC_ENCODE(x) (((uint64_t)(x)) & 0x3FFFF) >> 0x4)
1294
1295 // make_smem_desc: 创建 WGMMMA 的 shared memory 矩阵描述符
1296 // 这里沿用原始 hgemm_wgmma 实现里的描述符字段约定:
1297 //   - base addr: 当前 shared memory tile 基址
1298 //   - leading offset: 16 bytes = 8 个 half * 2B, 对应 WGMMMA 在 tile 内沿 minor
1299 //     方向前进一次的 byte 步长
1300 //   - stride offset: 1024 bytes = 64 * 8 * 2B, 对应跨到下一条 major stripe 的 byte 步长
1301 //   - bit 62: 打开 128B swizzle
1302 // 这些字段是当前 WGMMMA/TMA 布局下的实现常量, 不要把它直接背成 BM/BK 的原始字节数公式。
1303 __device__ inline uint64_t make_smem_desc(half *ptr) {
1304     uint32_t addr = static_cast<uint32_t>((__cvta_generic_to_shared(ptr)));
1305     uint64_t desc = 0x0000000000000000;
1306     desc |= SMEM_DESC_ENCODE(addr);
1307     desc |= SMEM_DESC_ENCODE((uint64_t)16) << 16; // leading dim offset bytes
1308     desc |= SMEM_DESC_ENCODE((uint64_t)1024) << 32; // stride dim offset bytes
1309     desc |= 1llu << 62; // 128B swizzle
1310     return desc;
1311 }
1312
1313 // ---- WGMMMA PTX 指令宏 ----
1314 // wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16
1315 // m64n128k16: M=64, N=128, K=16
1316 // f16.f16.f16: A=f16, B=f16, D=f16 (accumulation in f16)
1317 // 每线程 32 个 uint32 输出: D=64×128=8192 half, warpgroup 128 线程分担,
1318 // 每线程 64 half = 32 uint32
1319 // descA/descB: shared memory 描述符 (由 make_smem_desc 生成)
1320 // ScaleD: 0=clear accum, 1=accumulate (用于 K 维迭代时累加)
1321 #define WGMMMA_M64N128K16_F16F16F16(d, sA, sB, ScaleD, ScaleA, ScaleB, TransA, \
1322     TransB) \
1323 { \
1324     uint64_t desc_a = make_smem_desc(&(sA)[0]); \
1325     uint64_t desc_b = make_smem_desc(&(sB)[0]); \
1326     asm volatile( \
1327         "{\n" \
1328         "wgmma.mma_async.sync.aligned.m64n128k16.f16.f16.f16 " \
1329         "{%0, %1, %2, %3, %4, %5, %6, %7, " \
1330         "%8, %9, %10, %11, %12, %13, %14, %15, " \

```

```

1331     " %16, %17, %18, %19, %20, %21, %22, %23, " \
1332     " %24, %25, %26, %27, %28, %29, %30, %31}, " \
1333     " %32, " \
1334     " %33, " \
1335     " %34, %35, %36, %37, %38;\n" \
1336     "}\n" \
1337     : "+r"((d)[0][0]), "+r"((d)[0][1]), "+r"((d)[0][2]), "+r"((d)[0][3]), \
1338     "+r"((d)[1][0]), "+r"((d)[1][1]), "+r"((d)[1][2]), "+r"((d)[1][3]), \
1339     "+r"((d)[2][0]), "+r"((d)[2][1]), "+r"((d)[2][2]), "+r"((d)[2][3]), \
1340     "+r"((d)[3][0]), "+r"((d)[3][1]), "+r"((d)[3][2]), "+r"((d)[3][3]), \
1341     "+r"((d)[4][0]), "+r"((d)[4][1]), "+r"((d)[4][2]), "+r"((d)[4][3]), \
1342     "+r"((d)[5][0]), "+r"((d)[5][1]), "+r"((d)[5][2]), "+r"((d)[5][3]), \
1343     "+r"((d)[6][0]), "+r"((d)[6][1]), "+r"((d)[6][2]), "+r"((d)[6][3]), \
1344     "+r"((d)[7][0]), "+r"((d)[7][1]), "+r"((d)[7][2]), "+r"((d)[7][3]) \
1345     : "l"(desc_a), "l"(desc_b), "n"(int32_t(ScaleD)), \
1346     "n"(int32_t(ScaleA)), "n"(int32_t(ScaleB)), "n"(int32_t(TransA)), \
1347     "n"(int32_t(TransB))); \
1348 }
1349
1350 // ---- TMA Shared Memory Layout ----
1351 // Multi-stage pipeline: K_STAGE 个 stage, 每个 stage 存储 A[BM×BK] + B[BK×BN]
1352 template <int BM, int BK, int QSIZE> struct WgmmaSMem {
1353     alignas(128) half A[BM * BK * QSIZE]; // A tile: row-major [BM, BK]
1354     alignas(128) half B[BK * BN * QSIZE]; // B tile: row-major [BK, BN] (即 col-major B^T)
1355 };
1356
1357 // ---- WGMMMA Kernel: Warp Specialization + TMA ----
1358 // 面试重点 — Warp Specialization:
1359 //   WG0 (128 threads): Producer — 用 TMA 异步加载 A/B 到 shared memory
1360 //   WG1 (128 threads): Consumer — 用 WGMMMA 做矩阵乘
1361 //   同步: cuda::barrier (CTA 级别), Producer 发 full 信号, Consumer 发 empty 信号
1362 //   K_STAGE=3: 3 个 stage, Consumer 滞后 Producer 最多 2 步
1363 //   Grid: ((N+127)/128/S, (M+127)/128, S), S=(N+2047)/2048, 3D block swizzle
1364 //   Block: (256, 1, 1), 2 warpgroups(Producer+Consumer)
1365 //   source: LeetCUDA/kernels/hgemm/wgmma/hgemm_wgmma_fp16acc_stages_tn.cu
1366
1367 using cde = cuda::device::experimental;
1368
1369 template <const int WGMMMA_M = 64, const int WGMMMA_N = 128,
1370         const int WGMMMA_K = 16, const int BM = 128, const int BN = 128,
1371         const int BK = 64, const int NUM_THREADS = 256, const int K_STAGE = 3,
1372         const bool BLOCK_SWIZZLE = false>
1373 __global__ void __launch_bounds__(NUM_THREADS)
1374     hgemm_wgmma_stages_tn(
1375         int M, int N, int K, half *C,
1376         const CUtensorMap *__restrict__ tensorMapA,
1377         const CUtensorMap *__restrict__ tensorMapB) {
1378
1379     // 注意: tensorMapA/tensorMapB 需要由 host 侧按当前 tile 布局预先创建;
1380     // 对 row-major [H,W] 矩阵, TMA shape 参数写的是 (W,H) 而不是 (H,W),
1381     // 这是 TMA descriptor 最容易背错的地方之一。notes 这里只保留 kernel 主体,
1382     // 不展开宿主侧 create_tensor_map 细节。
1383
1384     const int bx = ((int)BLOCK_SWIZZLE) * blockIdx.z * gridDim.x + blockIdx.x;
1385     const int by = blockIdx.y;
1386     constexpr int num_consumers = (NUM_THREADS / 128) - 1; // 1 consumer WG
1387     constexpr int B_WG_M = BM / num_consumers; // 128
1388
1389     if (bx >= div_ceil(N, BN) || by >= div_ceil(M, BM))
1390         return;
1391
1392     extern __shared__ __align__(128) uint8_t smem[];
1393     WgmmaSMem<BM, BN, BK, K_STAGE> &s =
1394         *reinterpret_cast<WgmmaSMem<BM, BN, BK, K_STAGE>*>(smem);
1395     half *s_a = s.A;
1396     half *s_b = s.B;
1397
1398     // CTA barrier: 同步 Producer 和 Consumer
1399     __shared__ cuda::barrier<cuda::thread_scope_block> full[K_STAGE];
1400     __shared__ cuda::barrier<cuda::thread_scope_block> empty[K_STAGE];

```

```

1401
1402 const int num_blocks_k = K / BK;
1403 const int wg_idx = threadIdx.x / 128; // 0=Producer, 1=Consumer
1404 const int tid = threadIdx.x % 128;    // 0~127 within warpgroup
1405
1406 // 初始化 barriers (仅 thread 0 执行)
1407 if (threadIdx.x == 0) {
1408     for (int i = 0; i < K_STAGE; ++i) {
1409         // 这里的 129 不是 "warp 数" 也不是 "线程块总线程数",
1410         // 而是这个 barrier 上每轮会 arrive 的参与者总数: 128 个 consumer 线程 + 1 个 producer 提交线程。
1411         init(&full[i], num_consumers * 128 + 1); // 128 consumer + 1 producer
1412         init(&empty[i], num_consumers * 128 + 1); // same
1413     }
1414     cde::fence_proxy_async_shared_cta();
1415 }
1416 __syncthreads();
1417
1418 // ===== Producer Warpgroup (WG0) =====
1419 if (wg_idx == 0) {
1420     if (tid == 0) { // 仅 producer 的 thread 0 执行 TMA 加载
1421         // TMA 是硬件 DMA 提交指令: 发起一次 descriptor+坐标提交后, 后续搬运由硬件异步完成,
1422         // 不需要整个 producer warpgroup 里的 128 个线程都参与拷贝。
1423         int qidx = 0;
1424         for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1425              ++block_k_iter, ++qidx) {
1426             if (qidx == K_STAGE)
1427                 qidx = 0;
1428
1429             // 等待 Consumer 释放此 stage (empty 信号)
1430             empty[qidx].wait(empty[qidx].arrive());
1431
1432             // TMA 2D 加载 A tile: coords = (k_offset, m_offset)
1433             cde::cp_async_bulk_tensor_2d_global_to_shared(
1434                 &s_a[qidx * BK * BM], tensorMapA, block_k_iter * BK, by * BM,
1435                 full[qidx]);
1436
1437             // TMA 2D 加载 B tile: coords = (k_offset, n_offset)
1438             cde::cp_async_bulk_tensor_2d_global_to_shared(
1439                 &s_b[qidx * BK * BN], tensorMapB, block_k_iter * BK, bx * BN,
1440                 full[qidx]);
1441
1442             // 通知 Consumer 此 stage 已准备好 (full 信号)
1443             cuda::device::barrier_arrive_tx(full[qidx], 1,
1444                 (BK * BN + BK * BM) * sizeof(half));
1445         }
1446     }
1447 }
1448 // ===== Consumer Warpgroup (WG1) =====
1449 else {
1450     // Consumer 初始时"准备就绪", arrive 到所有 empty barriers
1451     for (int i = 0; i < K_STAGE; ++i) {
1452         empty[i].arrive();
1453     }
1454
1455     // 累加器: B_WG_M/WGMMMA_M=2 行 × WGMMMA_N/16=8 列 = 16 组寄存器
1456     uint32_t d[B_WG_M / WGMMMA_M][WGMMMA_N / 16][4] = {};
1457
1458     int qidx = 0;
1459     for (int block_k_iter = 0; block_k_iter < num_blocks_k;
1460          ++block_k_iter, ++qidx) {
1461         if (qidx == K_STAGE)
1462             qidx = 0;
1463
1464         // 等待 Producer 的 full 信号
1465         full[qidx].wait(full[qidx].arrive());
1466
1467         // WGMMMA 指令序列的常见记法是:
1468         // FENCE -> 发射一串 WGMMMA -> COMMIT_GROUP -> WAIT_GROUP -> 下一轮再 FENCE
1469         // fence 是为后续 WGMMMA 建立可见性/顺序关系; commit 表示当前这组 WGMMMA 已经发完;
1470         // wait 才是真正等待已 commit 的异步矩阵乘完成。

```

```

1471 // WGMMMA fence: 确保 smem 写可见、accum 寄存器准备好
1472 WGMMMA_FENCE();
1473
1474 // M 维迭代: BM/WGMMMA_M = 128/64 = 2
1475 #pragma unroll
1476 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1477     half *wgmma_sA = s_a + qidx * BK * BM + BK * m_it * WGMMMA_M;
1478
1479     // K 维迭代: BK/WGMMMA_K = 64/16 = 4
1480 #pragma unroll
1481     for (int k_it = 0; k_it < BK / WGMMMA_K; ++k_it) {
1482         WGMMMA_M64N128K16_F16F16F16(d[m_it], wgmma_sA + k_it * WGMMMA_K,
1483                                     s_b + qidx * BK * BN + k_it * WGMMMA_K,
1484                                     1, // ScaleD=1: accumulate (不清零)
1485                                     1, 1, 0, 0);
1486     }
1487 }
1488
1489 WGMMMA_COMMIT_GROUP();
1490 WGMMMA_WAIT_GROUP(0); // 等待所有 WGMMMA 完成
1491
1492 // 释放此 stage (发 empty 信号给 Producer)
1493 empty[qidx].arrive();
1494 }
1495
1496 // ===== Epilogue: 写回 C =====
1497 // WGMMMA m64n128k16 的输出寄存器映射可这样记:
1498 //   row = warp * 16 + lane / 4
1499 //   col = g * 16 + 2 * (lane % 4)
1500 //   d[m_it][g][0] -> (row, col)
1501 //   d[m_it][g][1] -> (row + 8, col)
1502 //   d[m_it][g][2] -> (row, col + 8)
1503 //   d[m_it][g][3] -> (row + 8, col + 8)
1504 // 所以每个线程一次写回 4 个 half2, 刚好覆盖当前 16x16 子块里的四个象限位置。
1505 const int lane = tid % 32;
1506 const int warp = tid / 32;
1507 const int row = warp * 16 + lane / 4;
1508 half *block_C = C + by * BM * N + bx * BN;
1509
1510 #pragma unroll
1511 for (int m_it = 0; m_it < B_WG_M / WGMMMA_M; ++m_it) {
1512     int yo = m_it * WGMMMA_M;
1513     #pragma unroll
1514     for (int g = 0; g < WGMMMA_N / 16; ++g) {
1515         int col = g * 16 + 2 * (lane % 4);
1516         // 将 uint32 直接 reinterpret 为 half2 pair 写入 C
1517         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col]) =
1518             d[m_it][g][0];
1519         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col]) =
1520             d[m_it][g][1];
1521         *reinterpret_cast<uint32_t*>(&block_C[(row + yo) * N + col + 8]) =
1522             d[m_it][g][2];
1523         *reinterpret_cast<uint32_t*>(&block_C[(row + yo + 8) * N + col + 8]) =
1524             d[m_it][g][3];
1525     }
1526 }
1527 }
1528 }
1529
1530 // =====
1531 // Phase 6: RoPE — 旋转位置编码 (Rotary Position Embedding)
1532 // =====
1533 // 面试题要点:
1534 //   - RoPE 数学公式: 对每对相邻维度做 2D 旋转
1535 //     [x1'] [cos(θ) -sin(θ)] [x1]
1536 //     [x2'] [sin(θ)  cos(θ)] [x2]
1537 //   - θ_i = 1 / (theta^(2i/d)), theta=10000.0f (Llama 风格)
1538 //   - token_pos = idx / N: token 在序列中的位置
1539 //   - token_idx = idx % N: token 内的维度对索引
1540 //   - 输入 [seq_len, hidden_size], 输出同形状

```

```

1541
1542 // Grid: ((seq_len * N + 255) / 256, 1, 1)
1543 // Block: (256, 1, 1)
1544 // source: LeetCUDA/kernels/rope/rope.cu
1545 __global__ void rope(float *x, float *out, int seq_len, int N) {
1546     int idx = blockIdx.x * blockDim.x + threadIdx.x;
1547     float x1 = x[idx * 2];
1548     float x2 = x[idx * 2 + 1];
1549     int token_pos = idx / N; // 序列位置
1550     int token_idx = idx % N; // 维度对索引
1551
1552     // 频率计算:  $\theta_i = 1 / (10000^{(2i/d)})$ 
1553     float exp_v = 1.0f / powf(10000.0f, 2 * token_idx / (N * 2.0f));
1554     float sin_v = sinf(token_pos * exp_v);
1555     float cos_v = cosf(token_pos * exp_v);
1556
1557     // 2D 旋转
1558     float out1 = x1 * cos_v - x2 * sin_v;
1559     float out2 = x1 * sin_v + x2 * cos_v;
1560     out[idx * 2] = out1;
1561     out[idx * 2 + 1] = out2;
1562 }
1563
1564 // =====
1565 // Phase 7: Mat Transpose — 矩阵转置 (Bank Conflict 专题)
1566 // =====
1567 // 面试题点 (Bank Conflict 专题):
1568 //   - Shared memory 有 32 个 bank, 每个 bank 4 bytes (32-bit)
1569 //   - 同一 warp 的多个线程访问同一 bank 的不同地址 → bank conflict
1570 //   - n-way bank conflict: n 个线程冲突 → 访问串行化为 n 次
1571 //   - 解决方案: PAD (在每行末尾加 1 个元素, 打破地址对齐)
1572 //
1573 // 转置的四步演进:
1574 //   naive: 非合并写入 (列优先写) → 每个 warp 产生 32 次内存事务
1575 //   shared: 写入 smem (行优先) → 从 smem 读取 (列优先) → 合并写入 gmem
1576 //   BCF: smem 布局 [WARP_SIZE_S*4][WARP_SIZE_S+PAD] = [64][17], PAD=1 加在第二维消除 bank conflict
1577 //   merge_write: 进一步将 4 次 separate store 合并为 1 次 float4 store
1578 // 注: 本文件仅实现 Level 1(naive) 与 Level 4(BCF+merge_write), Level 2/3 省略
1579
1580 // ---- Level 1: 基础版 (2D 索引, 合并读 + 非合并写) ----
1581 // 每个线程处理 1 个元素, block(16,16)
1582 // 读: x 按行优先访问, warp 的 32 线程访问 32 个连续地址 → 合并读取 ✓
1583 // 写: y 按列优先写入, 32 线程跨 row 行分散 → 非合并写入 x (32 次内存事务)
1584 // Grid: ((col + 15) / 16, (row + 15) / 16, 1), 每线程 1 元素
1585 // Block: (16, 16, 1)
1586 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1587 __global__ void mat_transpose(float *x, float *y, const int row, const int col) {
1588     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1589     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1590     if (global_y < row && global_x < col) {
1591         // 读取: x[global_x][global_y] = x[global_x * col + global_y], 行优先, 合并
1592         // 写入: y[global_y][global_x] = y[global_y * row +
1593         // global_x], 列优先, 非合并
1594         y[global_y * row + global_x] = x[global_x * col + global_y];
1595     }
1596 }
1597
1598 // ---- Level 4: 最佳版 (shared memory + bank conflict fix + merge write) ----
1599
1600 // Grid: ((col + 15) / 16, (row + 63) / 64, 1), 每线程 4 元素(float4)
1601 // Block: (16, 16, 1)
1602 // 注意: 该版本默认按 float4 打包写回; 最适合 row 能按 4 对齐的场景
1603 // source: LeetCUDA/kernels/mat-transpose/mat_transpose.cu
1604 __global__ void mat_transpose_padded(
1605     float *x, float *y, const int row, const int col) {
1606     const int global_x = blockIdx.x * blockDim.x + threadIdx.x;
1607     const int global_y = blockIdx.y * blockDim.y + threadIdx.y;
1608     const int local_x = threadIdx.x;
1609     const int local_y = threadIdx.y;

```



```

1611 constexpr int WARP_SIZE_S = 16;
1612 constexpr int PAD = 1;
1613 // Bank conflict fix: 每行多 1 个元素, 打破 32-bank 对齐
1614 __shared__ float tile[WARP_SIZE_S * 4][WARP_SIZE_S + PAD];
1615
1616 if (global_y * 4 < row && global_x < col) {
1617     // Step 1: 从 global memory 读取 4 行, 写入 shared
1618     // memory (行优先, 合并读取)
1619     float4 x_val;
1620     x_val.x = x[(global_y * 4) * col + global_x];
1621     x_val.y = x[(global_y * 4 + 1) * col + global_x];
1622     x_val.z = x[(global_y * 4 + 2) * col + global_x];
1623     x_val.w = x[(global_y * 4 + 3) * col + global_x];
1624     tile[local_y * 4][local_x] = x_val.x;
1625     tile[local_y * 4 + 1][local_x] = x_val.y;
1626     tile[local_y * 4 + 2][local_x] = x_val.z;
1627     tile[local_y * 4 + 3][local_x] = x_val.w;
1628     __syncthreads();
1629
1630     // Step 2: 从 shared memory 读取 ("转置"方向), 合并写入 global memory
1631     float4 smem_val;
1632     smem_val.x = tile[local_x * 4][local_y];
1633     smem_val.y = tile[local_x * 4 + 1][local_y];
1634     smem_val.z = tile[local_x * 4 + 2][local_y];
1635     smem_val.w = tile[local_x * 4 + 3][local_y];
1636
1637     const int gid_x = blockIdx.x * blockDim.x;
1638     const int gid_y = blockIdx.y * blockDim.y * 4;
1639     const int out_y = gid_y + local_x * 4;
1640     const int out_x = gid_x + local_y;
1641
1642     // float4 merge write: 1 次 128-bit store; 这里也隐含 out_y 为 4 的倍数
1643     *reinterpret_cast<float4*>(&y[(out_x * row + out_y) / 4]) =
1644         *reinterpret_cast<float4*>(&smem_val);
1645 }
1646 }
1647
1648 // =====
1649 // Phase 8: 杂项 Ops — Dot Product / Block All Reduce / Histogram
1650 // =====
1651 // 面试要点:
1652 //   - Dot Product: elementwise mul + reduce, 演示两种 reduce 模式结合
1653 //   - Block All Reduce: 多 block 各自做 reduce, 最后 atomicAdd 到全局结果
1654 //   - Histogram: global atomicAdd 模式, 演示多线程竞争同一 bin 的原子更新
1655
1656 // ---- Dot Product: y = sum(a[i] * b[i]) ----
1657 // 核心模式: elementwise 乘法 → block reduce → atomicAdd 全局累加
1658 template <const int NUM_THREADS = 128>
1659 // Grid: ((N + 127) / 128, 1, 1)
1660 // Block: (128, 1, 1)
1661 // source: LeetCode/kernels/dot-product/dot_product.cu
1662 __global__ void dot(float *a, float *b, float *y, int N) {
1663     int tid = threadIdx.x;
1664     int idx = blockIdx.x * NUM_THREADS + tid;
1665     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1666     __shared__ float reduce_smem[NUM_WARPS];
1667
1668     float prod = (idx < N) ? a[idx] * b[idx] : 0.0f;
1669     int warp = tid / WARP_SIZE;
1670     int lane = tid % WARP_SIZE;
1671
1672     prod = warp_reduce_sum<WARP_SIZE>(prod);
1673     if (lane == 0)
1674         reduce_smem[warp] = prod;
1675     __syncthreads();
1676
1677     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1678     if (warp == 0) // 只需要 warp 0 的线程继续 reduce 即可
1679         prod = warp_reduce_sum<NUM_WARPS>(prod);
1680     if (tid == 0)

```

```

1681     atomicAdd(y, prod);
1682 }
1683
1684 // Dot Product + float4
1685 template <const int NUM_THREADS = 128 / 4>
1686 // Grid: ((N + 127) / 128, 1, 1)
1687 // Block: (32, 1, 1), 128/4=32
1688 // 注意: 该版本默认输入地址满足 float4 对齐; 最适合 N 按 4 对齐的场景
1689 // source: LeetCode/kernels/dot-product/dot_product.cu
1690 __global__ void dot_vec4(float *a, float *b, float *y, int N) {
1691     int tid = threadIdx.x;
1692     int idx = (blockIdx.x * NUM_THREADS + tid) * 4;
1693     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1694     __shared__ float reduce_smem[NUM_WARPS];
1695
1696     float4 reg_a = FLOAT4(a[idx]);
1697     float4 reg_b = FLOAT4(b[idx]);
1698     float prod = (idx < N) ? (reg_a.x * reg_b.x + reg_a.y * reg_b.y +
1699                             reg_a.z * reg_b.z + reg_a.w * reg_b.w)
1700                       : 0.0f;
1701     int warp = tid / WARP_SIZE;
1702     int lane = tid % WARP_SIZE;
1703
1704     prod = warp_reduce_sum<WARP_SIZE>(prod);
1705     if (lane == 0)
1706         reduce_smem[warp] = prod;
1707     __syncthreads();
1708
1709     prod = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1710     if (warp == 0)
1711         prod = warp_reduce_sum<NUM_WARPS>(prod);
1712     if (tid == 0)
1713         atomicAdd(y, prod);
1714 }
1715
1716 // ---- Block Reduce Sum All: y = sum(a[0..N-1]) ----
1717 // 多 block 各自做 warp-smem->warp0 reduce, 然后 atomicAdd 到全局 y
1718 // 跨 block 求和的常见模式, 适合 N 较大时使用;
1719 template <const int NUM_THREADS = 128>
1720 // Grid: ((N + 127) / 128, 1, 1)
1721 // Block: (128, 1, 1)
1722 // source: LeetCode/kernels/reduce/block_all_reduce.cu
1723 __global__ void block_reduce_v2(float *a, float *y, int N) {
1724     int tid = threadIdx.x;
1725     int idx = blockIdx.x * NUM_THREADS + tid;
1726     constexpr int NUM_WARPS = (NUM_THREADS + WARP_SIZE - 1) / WARP_SIZE;
1727     __shared__ float reduce_smem[NUM_WARPS];
1728
1729     float sum = (idx < N) ? a[idx] : 0.0f;
1730     int warp = tid / WARP_SIZE;
1731     int lane = tid % WARP_SIZE;
1732
1733     sum = warp_reduce_sum<WARP_SIZE>(sum);
1734     if (lane == 0)
1735         reduce_smem[warp] = sum;
1736     __syncthreads();
1737
1738     sum = (lane < NUM_WARPS) ? reduce_smem[lane] : 0.0f;
1739     if (warp == 0)
1740         sum = warp_reduce_sum<NUM_WARPS>(sum);
1741     if (tid == 0)
1742         atomicAdd(y, sum);
1743 }
1744
1745 // ---- Histogram: y[a[i]]++ ----
1746 // 演示 atomicAdd 的用法: 多个线程可能同时更新同一个 bin
1747 // Grid: ((N + 255) / 256, 1, 1)
1748 // Block: (256, 1, 1)
1749 // source: LeetCode/kernels/histogram/histogram.cu
1750 __global__ void histogram(int *a, int *y, int N) {

```

```

1751 int idx = blockIdx.x * blockDim.x + threadIdx.x;
1752 if (idx < N)
1753     atomicAdd(&(y[a[idx]]), 1);
1754 }
1755
1756 // =====
1757 // Phase 9: FlashAttention-2 (Split-Q + MMA m16n8k16)
1758 // =====
1759 // 面试题点 (FlashAttention 算法) :
1760 // 1. 核心问题: 标准 Attention 的  $O(N^2)$  中间矩阵 ( $S=QK^T$ ) 必须写入 HBM,
1761 //    但 HBM 带宽是瓶颈 → FlashAttention 用 tiling + online softmax 避免写回
1762 // 2. FA 三板斧:
1763 //    a) Tiling: Q 分块 [Br,d], K/V 沿 seqLen 分块 [Bc,d]
1764 //    b) Online Softmax: 迭代更新 m(行max) 和 l(行sum), 无需全局同步
1765 //    c) Recomputation(反向): 反向传播时重新计算 S/P, 而非存储中间矩阵
1766 // 3. Split-Q 设计: 所有 warp 共享同一块 K, 各 warp 处理 Q 的不同行片段
1767 //    - warp_KV=0 (所有 warp 共享 K), warp_QP=warp_id (各 warp 不同 Q 行)
1768 //    - 优点: 减少 warp 间通信和 shuffle
1769 // 4. Online rescaling 公式 (FA 核心, arXiv:2307.08691) :
1770 //    for each K,V tile:
1771 //        S_cur = Q @ K^T // 未缩放, 存入 R_S
1772 //        m_new = max(m_old, row_max(S_cur * scale))
1773 //        P_cur = exp(S_cur * scale - m_new) // ← 写回 R_S 寄存器!
1774 //        l_new = exp(m_old - m_new) * l_old + row_sum(P_cur)
1775 //        O_new = diag(exp(m_old - m_new)) * O_old + P_cur @ V
1776 //        O_final = O_new / l_final
1777 // 5. 为什么 R_S 可以直接用作 P@V 的 A 矩阵?
1778 //    - R_S 经过 softmax 后, 存储的是  $P = \exp(S - m)$ , 数据仍然是 half 精度
1779 //    - 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局, 使 softmax
1780 //      后的 P 可以继续留在 R_S 中供后面的 P@V 直接消费
1781 //    - 这是此实现的寄存器布局复用技巧, 不要背成“所有 MMA A/C fragment
1782 //      都天然同构”的通用结论
1783 //
1784 // 本实现参考: FlashAttention-2 (Dao et al., arXiv:2307.08691)
1785 // 从 LeetCUDA flash-attn/mma/basic/flash_attn_mma_split_q.cu 提取
1786 // Grid: ((QKV_seqLen + 63) / 64, QKV_batch * QKV_head, 1), Br=64
1787 // Block: (128, 1, 1), kNumThreads=WARP_SIZE*kMmaTileSeqLenQ*kMmaTileSeqLenK=128
1788 // source: LeetCUDA/kernels/flash-attn/mma/basic/flash_attn_mma_split_q.cu
1789
1790 // ---- 寄存器填充辅助函数 ----
1791 template <typename T, int M, const int N, const int K = 2>
1792 __device__ inline void fill_3D_regs(T (&R)[M][N][K], T val) {
1793     #pragma unroll
1794     for (int i = 0; i < M; ++i)
1795         #pragma unroll
1796         for (int j = 0; j < N; ++j)
1797             #pragma unroll
1798             for (int k = 0; k < K; ++k)
1799                 R[i][j][k] = val;
1800 }
1801
1802 template <typename T, int M, const int N = 2>
1803 __device__ inline void fill_2D_regs(T (&R)[M][N], T val) {
1804     #pragma unroll
1805     for (int i = 0; i < M; ++i)
1806         #pragma unroll
1807         for (int j = 0; j < N; ++j)
1808             R[i][j] = val;
1809 }
1810
1811 // =====
1812 // FlashAttention-2 Split-Q Kernel (完整实现)
1813 // =====
1814 // Q,K,V,0: [batch_size, num_heads, seq_len, head_dim], [B,H,N,d]
1815 //
1816 // Tile 设计 (以 kHeadDim=64 为例) :
1817 // Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ = 16*4*1 = 64
1818 // Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK = 8*1*8 = 64
1819 // Warp 布局: 4 warps, warp_QP 0~3 各处理 16 行, warp_KV=0 共享 K
1820 //

```

```

1821 // 执行流程:
1822 // 1) 预加载 Q[Br,d] 到 smem (只加载一次, split-Q 的核心优势)
1823 // 2) 外循环: 沿 K seqlen 分块迭代 (Tc = seqlen/Bc)
1824 // 3) 3a: cp.async 加载当前 V + 预加载下一个 K (多 stage pipeline)
1825 // 4) 3b: QK^T — 沿 head dim 内循环 → ldmatrix Q/K → HMMA16816
1826 // 5) 3c: Online Safe Softmax — warp reduce max/row → exp(S*scale - max)
1827 // → 关键: 将 P 写回 R_S 寄存器 (替换 S), R_S 现在存储 P matrix
1828 // 6) 3d: P@V — 沿 V_Bc 内循环 → ldmatrix V (transposed) → 直接用 R_S 做 A
1829 // → 当前实现依赖同一组寄存器布局约定, 避免为 P@V 额外重组 P fragment
1830 // 7) 3e: Online rescaling — O_new = exp(m_old-m_new)*O_old + P@V
1831 // 8) 最终 rescale: O_final = (1/l_final) * O_final
1832 // 9) Epilogue: warp shuffle + 128-bit collective store
1833
1834 template <
1835     const int kHeadDim,          // head dim: 32, 64, 128
1836     const int kMmaAtomM,         // 16 (MMA instruction M dimension)
1837     const int kMmaAtomN,         // 8 (MMA instruction N dimension)
1838     const int kMmaAtomK,         // 16 (MMA instruction K dimension)
1839     const int kMmaTileSeqLenQ,   // MMA tiles along Q's M dim, 4 → Br=16*4=64
1840     const int kMmaTileSeqLenK,   // MMA tiles along K's N dim, 1 → Bc basis=8
1841     const int kMmaTileSeqLenP,   // MMA tiles for P@V M dim, must equal
1842                                   // kMmaTileSeqLenQ
1843     const int kMmaTileHeadDimV,  // MMA tiles for P@V N dim (head dim direction)
1844     const int kWarpTileSeqLenQ,  // warp tiles along Q's M, 1 → Br per warp=16
1845     const int kWarpTileSeqLenK,  // warp tiles along K's N, 8 → Bc_warp=8*8=64
1846     const int kWarpTileSeqLenP,  // warp tiles for P@V M dim, 1
1847     const int kWarpTileHeadDimV, // warp tiles for P@V N dim,
1848                                   // kHeadDim/(8*kMmaTileHeadDimV)
1849     const int kStage,            // pipeline stages for K: 1 or 2
1850     const int kPad>             // padding for bank conflict avoidance
1851 __global__ void __launch_bounds__(WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK)
1852     flash_attn_mma_stages_split_q(half *Q, half *K, half *V, half *O,
1853                                   int QKV_seqlen, int QKV_head) {
1854
1855     // Tile dimensions
1856     constexpr int Br = kMmaAtomM * kMmaTileSeqLenQ * kWarpTileSeqLenQ; // 64
1857     constexpr int Bc = kMmaAtomN * kMmaTileSeqLenK * kWarpTileSeqLenK; // 64
1858     constexpr int kNumThreads =
1859         WARP_SIZE * kMmaTileSeqLenQ * kMmaTileSeqLenK; // 128
1860     const int Tc = (QKV_seqlen + Bc - 1) / Bc;
1861     // 原始实现默认 seqlen 与 Bc 对齐; 最后一个不完整 tile 需要额外 pad/边界处理。
1862     // 这里保留 ceil 写法是为了说明 tile 划分方式, 不等于当前实现已经完整处理了尾 tile。
1863     const float scale = 1.0f / sqrtf((float)kHeadDim);
1864
1865     // Block indexing
1866     const int QKV_batch_id = blockIdx.y / QKV_head;
1867     const int QKV_head_id = blockIdx.y % QKV_head;
1868     const int Q_tile_id = blockIdx.x;
1869     const int tid = threadIdx.x;
1870     const int warp_id = tid / WARP_SIZE;
1871     const int lane_id = tid % WARP_SIZE;
1872     const int warp_QP = warp_id; // Split-Q: 每个 warp 处理不同的 Q 行片段
1873     const int warp_KV = 0; // 所有 warp 共享 K (减少跨 warp 通信)
1874
1875     // Global memory base offsets for this (batch, head)
1876     // 这里默认 Q/K/V 共享同一 per-head 基址布局, 对应 self-attention 场景
1877     const int Q_gmem_offset =
1878         (QKV_batch_id * QKV_head * QKV_seqlen + QKV_head_id * QKV_seqlen) *
1879         kHeadDim;
1880     const int K_gmem_offset = Q_gmem_offset;
1881     const int V_gmem_offset = Q_gmem_offset;
1882     const int O_gmem_offset = Q_gmem_offset;
1883
1884     // Thread-to-smem mapping for cooperative load
1885     int load_smem_Q_Br = tid / (kNumThreads / Br);
1886     int load_smem_Q_d =
1887         (tid % (kNumThreads / Br)) * (kHeadDim / (kNumThreads / Br));
1888     int load_smem_K_Bc = tid / (kNumThreads / Bc);
1889     int load_smem_K_d =
1890         (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));

```

```

1891 int load_smem_V_Bc = tid / (kNumThreads / Bc);
1892 int load_smem_V_d =
1893     (tid % (kNumThreads / Bc)) * (kHeadDim / (kNumThreads / Bc));
1894
1895 int load_gmem_Q_Br = Q_tile_id * Br + load_smem_Q_Br;
1896 if (load_gmem_Q_Br >= QKV_seqlen)
1897     return;
1898
1899 // ---- Shared memory layout ----
1900 extern __shared__ half smem[];
1901 constexpr int Q_tile_size = Br * (kHeadDim + kPad); // Q tile: [Br, d+kPad]
1902 constexpr int KV_tile_size = Bc * (kHeadDim + kPad); // K/V tile: [Bc, d+kPad]
1903 half *Q_tile_smem = smem;
1904 half *K_tile_smem = Q_tile_smem + Q_tile_size;
1905 half *V_tile_smem = K_tile_smem + KV_tile_size; // kStage copies of K
1906 // 原始 kernel 还留了一个优化点: 若 kStage=1, K 和 V 在时序上并不重叠,
1907 // 理论上可以复用同一块 KV shared memory 来进一步压缩 smem 占用。
1908
1909 uint32_t smem_Q_base_ptr = __cvta_generic_to_shared(Q_tile_smem);
1910 uint32_t smem_K_base_ptr = __cvta_generic_to_shared(K_tile_smem);
1911 uint32_t smem_V_base_ptr = __cvta_generic_to_shared(V_tile_smem);
1912
1913 // ---- Online Softmax persistent state ----
1914 // lane_block_row_max_old[i][r]: running max for row r of warp tile i
1915 // lane_block_row_sum_old[i][r]: running denominator l for row r of warp tile
1916 // i
1917 float lane_block_row_max_old[kWarpTileSeqLenQ][2];
1918 float lane_block_row_sum_old[kWarpTileSeqLenQ][2];
1919 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_max_old, -INFINITY);
1920 fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_block_row_sum_old, 0.0f);
1921
1922 // ---- Register allocation ----
1923 uint32_t R_Q[kWarpTileSeqLenQ][4]; // Q regs
1924 uint32_t R_K[kWarpTileSeqLenK][2]; // K regs
1925 uint32_t R_V[kWarpTileHeadDimV][2]; // V regs
1926 // R_S / R_O / R_D 都按 mma.sync.aligned.m16n8k16 的 fragment 约定存储。
1927 // 对单个 m16n8k16 tile 而言:
1928 // - reg[0] 持有该 tile 前 8 行里的两个 half 值
1929 // - reg[1] 持有该 tile 后 8 行里的两个 half 值
1930 // 后续 softmax、P@V、online rescale 都直接围绕这组 fragment 布局做寄存器内变换。
1931 uint32_t R_S[kWarpTileSeqLenQ][kWarpTileSeqLenK][2]; // S=Q@K^T / P=softmax(S)
1932 uint32_t R_O[kWarpTileSeqLenP][kWarpTileHeadDimV][2]; // O for current tile
1933 uint32_t R_D[kWarpTileSeqLenP][kWarpTileHeadDimV]
1934     [2]; // O accumulator (final output)
1935
1936 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
1937 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_D, 0);
1938
1939 // =====
1940 // Step 1: 加载 Q[Br, d] 到 shared memory (整个外循环只加载一次)
1941 // =====
1942 {
1943     int load_gmem_Q_addr =
1944         Q_gmem_offset + load_gmem_Q_Br * kHeadDim + load_smem_Q_d;
1945     uint32_t load_smem_Q_ptr =
1946         smem_Q_base_ptr +
1947         (load_smem_Q_Br * (kHeadDim + kPad) + load_smem_Q_d) * sizeof(half);
1948 #pragma unroll
1949     for (int i = 0; i < (kHeadDim / (kNumThreads / Br)); i += 8) {
1950         CP_ASYNC_CG(load_smem_Q_ptr + i * 2, &Q[load_gmem_Q_addr + i], 16);
1951     }
1952     CP_ASYNC_COMMIT_GROUP();
1953 }
1954
1955 // =====
1956 // Step 2: 预加载前 (kStage-1) 个 K tile (多 stage pipeline 预热)
1957 // 注意: Q 由 blockIdx.x 固定到当前 Q tile; 而 K/V 的 seqlen 遍历始终从 tile 0 开始,
1958 // 后续在外循环里不断递增到 tile 1/2/3/.../Tc-1。
1959 // =====
1960 if constexpr (kStage > 1) {

```

```

1961 #pragma unroll
1962 for (int stage = 0; stage < (kStage - 1); ++stage) {
1963     int load_gmem_K_Bc = stage * Bc + load_smem_K_Bc;
1964     int load_gmem_K_addr =
1965         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
1966     uint32_t load_smem_K_ptr =
1967         smem_K_base_ptr +
1968         (stage * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
1969          load_smem_K_d) *
1970         sizeof(half);
1971 #pragma unroll
1972 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
1973     CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
1974 }
1975 CP_ASYNC_COMMIT_GROUP();
1976 }
1977 CP_ASYNC_WAIT_GROUP(kStage - 2);
1978 __syncthreads();
1979 }
1980
1981 // =====
1982 // Step 3: 外循环 — 沿 K seqlen 迭代 (Tc = ceil(seqlen/Bc))
1983 // 每次迭代处理一个 K[Bc,d] + V[Bc,d] tile
1984 // =====
1985 #pragma unroll 1
1986 for (int tile_K_seqlen = 0; tile_K_seqlen < Tc; ++tile_K_seqlen) {
1987     int smem_sel = tile_K_seqlen % kStage;
1988     int smem_sel_next = (tile_K_seqlen + (kStage - 1)) % kStage;
1989
1990     // ---- 3a: 异步加载 K/V tile (多 stage pipeline) ----
1991     if constexpr (kStage > 1) {
1992         // 只有 kStage>1 才能真正做 K 的 pipeline:
1993         // smem_sel 负责“当前正在计算”的 K tile, smem_sel_next 负责“下一轮预取”的 K tile。
1994         // 若 kStage=1, 这两个槽位永远都等于 0, 当前 K 还没算完就无法安全覆盖同一块 smem。
1995         // Load current V tile (no pipeline for V — one stage is enough)
1996         {
1997             int load_gmem_V_Bc = tile_K_seqlen * Bc + load_smem_V_Bc;
1998             int load_gmem_V_addr =
1999                 V_gmem_offset + load_gmem_V_Bc * kHeadDim + load_smem_V_d;
2000             uint32_t load_smem_V_ptr =
2001                 smem_V_base_ptr +
2002                 (load_smem_V_Bc * (kHeadDim + kPad) + load_smem_V_d) * sizeof(half);
2003 #pragma unroll
2004 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2005     CP_ASYNC_CG(load_smem_V_ptr + i * 2, &V[load_gmem_V_addr + i], 16);
2006 }
2007 CP_ASYNC_COMMIT_GROUP();
2008 }
2009
2010 // Prefetch next K tile (pipelined)
2011 if ((tile_K_seqlen + 1) < Tc) {
2012     int load_gmem_K_Bc = (tile_K_seqlen + 1) * Bc + load_smem_K_Bc;
2013     int load_gmem_K_addr =
2014         K_gmem_offset + load_gmem_K_Bc * kHeadDim + load_smem_K_d;
2015     uint32_t load_smem_K_ptr =
2016         smem_K_base_ptr +
2017         (smem_sel_next * KV_tile_size + load_smem_K_Bc * (kHeadDim + kPad) +
2018          load_smem_K_d) *
2019         sizeof(half);
2020 #pragma unroll
2021 for (int i = 0; i < (kHeadDim / (kNumThreads / Bc)); i += 8) {
2022     CP_ASYNC_CG(load_smem_K_ptr + i * 2, &K[load_gmem_K_addr + i], 16);
2023 }
2024 CP_ASYNC_COMMIT_GROUP();
2025 }
2026 }
2027
2028 // ---- 3b: Q@K^T = S[Br, Bc] — 沿 head dim (d/kMmaAtomK=16) 内循环 ----
2029 fill_3D_regs<uint32_t, kWarpTileSeqLenQ, kWarpTileSeqLenK, 2>(R_S, 0);
2030 #pragma unroll

```

```

2031     for (int tile_K_d = 0; tile_K_d < (kHeadDim / kMmaAtomK); ++tile_K_d) {
2032         // ldmatrix.x4: 加载 Q 的 m16k16 片段到 R_Q
2033     #pragma unroll
2034     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2035         int warp_smem_Q_Br =
2036             warp_QP * (kMmaAtomM * kWarpTileSeqLenQ) + i * kMmaAtomM;
2037         int lane_smem_Q_Br =
2038             warp_smem_Q_Br + lane_id % 16; // ldmatrix uses 16 lanes
2039         int lane_smem_Q_d = tile_K_d * kMmaAtomK + (lane_id / 16) * 8; // 0, 8
2040         uint32_t lane_smem_Q_ptr =
2041             smem_Q_base_ptr +
2042             (lane_smem_Q_Br * (kHeadDim + kPad) + lane_smem_Q_d) * sizeof(half);
2043         LDMATRIX_X4(R_Q[i][0], R_Q[i][1], R_Q[i][2], R_Q[i][3],
2044             lane_smem_Q_ptr);
2045     }
2046
2047     // ldmatrix.x2: 加载 K 的 k16n8 片段到 R_K
2048     // K[Bc,d] row-major = K^T[d,Bc] col-major (NT 布局的 B 矩阵)
2049     #pragma unroll
2050     for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2051         int warp_smem_K_Bc =
2052             warp_KV * (kMmaAtomN * kWarpTileSeqLenK) + j * kMmaAtomN;
2053         int lane_smem_K_Bc =
2054             warp_smem_K_Bc + lane_id % 8; // ldmatrix B uses 8 lanes
2055         int lane_smem_K_d =
2056             tile_K_d * kMmaAtomK + ((lane_id / 8) % 2) * 8; // 0, 8
2057         uint32_t lane_smem_K_ptr =
2058             smem_K_base_ptr +
2059             (smem_sel * KV_tile_size + lane_smem_K_Bc * (kHeadDim + kPad) +
2060             lane_smem_K_d) *
2061             sizeof(half);
2062         LDMATRIX_X2(R_K[j][0], R_K[j][1], lane_smem_K_ptr);
2063     }
2064
2065     // MMA: S[tile] += Q[tile] @ K^T[tile]
2066     #pragma unroll
2067     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2068     #pragma unroll
2069         for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2070             HMMA16816(R_S[i][j][0], R_S[i][j][1], R_Q[i][0], R_Q[i][1], R_Q[i][2],
2071                 R_Q[i][3], R_K[j][0], R_K[j][1], R_S[i][j][0],
2072                 R_S[i][j][1]);
2073         }
2074     } // end loop over d
2075     __syncthreads();
2076
2077     // =====
2078     // 3c: Online Safe Softmax — row-wise max + exp + sum, then store P back to
2079     // R_S
2080     // =====
2081     // MMA C fragment layout for m16n8k16 (PTX ISA 对应的线程寄存器分布):
2082     //   - R_S[i][j][0] 对应当前 16x8 tile 的 rows 0~7 里的两个 half 值 {c0,c1}
2083     //   - R_S[i][j][1] 对应 rows 8~15 里的两个 half 值 {c2,c3}
2084     //   - lane 0~3 持有 row 0 的片段, lane 4~7 持有 row 1, ..., lane 28~31 持有 row 7
2085     //   - 对于 rows 8~15 也是同样的 lane 分组, 只是读取的是 reg[1]
2086     // 这就是为什么后面做 row max / row sum 时, warp 内真正参与同一行归约的是
2087     // {0,4,8,...,28} 这一类 4-lane 子组, 而不是整 warp 32 个线程。
2088     // Each (i, j) pair = one 16x8 MMA tile; there are kWarpTileSeqLenQ x
2089     // kWarpTileSeqLenK tiles.
2090
2091     float lane_row_max_new[kWarpTileSeqLenQ][2];
2092     float lane_row_sum_new[kWarpTileSeqLenQ][2];
2093     fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_max_new, -INFINITY);
2094     fill_2D_regs<float, kWarpTileSeqLenQ, 2>(lane_row_sum_new, 0.0f);
2095
2096     // ---- Pass 1: Thread-level reduce max across all columns (kWarpTileSeqLenK
2097     // tiles) ----
2098     #pragma unroll
2099     for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2100

```



```

2101 #pragma unroll
2102 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2103     // Extract half values from R_S registers (C matrix fragment layout)
2104     float2 t_reg_S_0 =
2105         __half22float2(HALF2(R_S[i][j][0])); // rows 0~7: {c0, c1}
2106     float2 t_reg_S_1 =
2107         __half22float2(HALF2(R_S[i][j][1])); // rows 8~15: {c2, c3}
2108     // S = (Q@K^T) * scale
2109     float tmp_max_0 = max(t_reg_S_0.x, t_reg_S_0.y) * scale;
2110     float tmp_max_1 = max(t_reg_S_1.x, t_reg_S_1.y) * scale;
2111     lane_row_max_new[i][0] = max(lane_row_max_new[i][0], tmp_max_0);
2112     lane_row_max_new[i][1] = max(lane_row_max_new[i][1], tmp_max_1);
2113 }
2114 // Warp-level reduce max (warp_size = 4 for Q@K^T — only lanes
2115 // {0,4,8,...,28} hold valid data)
2116 lane_row_max_new[i][0] =
2117     warp_reduce_max<float, 4>(lane_row_max_new[i][0]);
2118 lane_row_max_new[i][1] =
2119     warp_reduce_max<float, 4>(lane_row_max_new[i][1]);
2120 }
2121
2122 // ---- Pass 2: Compute P = exp(S*scale - m_new), store back to R_S ----
2123 // 面试关键点: 这里将 P 写回 R_S 寄存器!
2124 // 为什么可以? 当前实现依赖 m16n8k16 这一路径下约定好的 fragment 布局,
2125 // 使 softmax 后的 P 能继续留在 R_S 中供后面的 P@V 直接消费, 无需额外重组。
2126 #pragma unroll
2127 for (int i = 0; i < kWarpTileSeqLenQ; ++i) {
2128     // m_new = max(m_old, m_cur)
2129     float block_row_max_new_0 =
2130         max(lane_block_row_max_old[i][0], lane_row_max_new[i][0]);
2131     float block_row_max_new_1 =
2132         max(lane_block_row_max_old[i][1], lane_row_max_new[i][1]);
2133
2134 #pragma unroll
2135 for (int j = 0; j < kWarpTileSeqLenK; ++j) {
2136     float2 t_reg_S_0 = __half22float2(HALF2(R_S[i][j][0]));
2137     float2 t_reg_S_1 = __half22float2(HALF2(R_S[i][j][1]));
2138
2139     // P = exp(S * scale - m_new), 用 fma 保证精度
2140     t_reg_S_0.x =
2141         __expf(__fmaf_rn(t_reg_S_0.x, scale, -block_row_max_new_0));
2142     t_reg_S_0.y =
2143         __expf(__fmaf_rn(t_reg_S_0.y, scale, -block_row_max_new_0));
2144     t_reg_S_1.x =
2145         __expf(__fmaf_rn(t_reg_S_1.x, scale, -block_row_max_new_1));
2146     t_reg_S_1.y =
2147         __expf(__fmaf_rn(t_reg_S_1.y, scale, -block_row_max_new_1));
2148
2149     // Accumulate row sums
2150     lane_row_sum_new[i][0] += (t_reg_S_0.x + t_reg_S_0.y);
2151     lane_row_sum_new[i][1] += (t_reg_S_1.x + t_reg_S_1.y);
2152
2153     // 关键: 将 P 写回 R_S! R_S 现在存储的是 P = softmax(S), 不是 S
2154     HALF2(R_S[i][j][0]) = __float22half2_rn(t_reg_S_0);
2155     HALF2(R_S[i][j][1]) = __float22half2_rn(t_reg_S_1);
2156 }
2157
2158 // Warp-level reduce sum (warp_size = 4, same as max)
2159 lane_row_sum_new[i][0] =
2160     warp_reduce_sum<float, 4>(lane_row_sum_new[i][0]);
2161 lane_row_sum_new[i][1] =
2162     warp_reduce_sum<float, 4>(lane_row_sum_new[i][1]);
2163 }
2164 __syncthreads();
2165
2166 // =====
2167 // 3d: P@V — P[Br,Bc] @ V[Bc,d] = O[Br,d]
2168 // =====
2169 // Wait for V to be ready before computing P@V
2170 if constexpr (kStage > 1) {

```



```

2171     if ((tile_K_seqlen + 1) < Tc) {
2172         CP_ASYNC_WAIT_GROUP(1);
2173     } else {
2174         CP_ASYNC_WAIT_GROUP(0);
2175     }
2176 } else {
2177     CP_ASYNC_WAIT_GROUP(0);
2178 }
2179 __syncthreads();
2180
2181 fill_3D_regs<uint32_t, kWarpTileSeqLenP, kWarpTileHeadDimV, 2>(R_0, 0);
2182
2183 // tile_V_Bc: iterate over chunks of Bc/K=16 columns in P matrix
2184 // Bc=kMmaAtomK=16 → 1 iteration for kHeadDim≤64 configurations
2185 #pragma unroll
2186 for (int tile_V_Bc = 0; tile_V_Bc < (Bc / kMmaAtomK); ++tile_V_Bc) {
2187     // ldmatrix.x2.trans: load V[Bc,d] with transposition
2188     // V is row-major [Bc,d], but NN matmul needs B matrix in col-major → use
2189     // transposed ldmatrix
2190 #pragma unroll
2191 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2192     int warp_smem_V_d =
2193         warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2194     int lane_smem_V_Bc = tile_V_Bc * kMmaAtomK + lane_id % 16;
2195     int lane_smem_V_d = warp_smem_V_d;
2196     uint32_t lane_smem_V_ptr =
2197         smem_V_base_ptr +
2198         (lane_smem_V_Bc * (kHeadDim + kPad) + lane_smem_V_d) * sizeof(half);
2199     LDMATRIX_X2_T(R_V[j][0], R_V[j][1], lane_smem_V_ptr);
2200 }
2201
2202 // P matrix layout in R_S[i][j][2]:
2203 //   MMA = m16n8k16, Br=16x4=64, Bc=8x8=64, layout: 4 warps
2204 //   | 64x64 | warp_KV 0 |
2205 //   | warp_QP 0 | MMA 0 ... MMA 0 (x8) |
2206 //   | warp_QP 1 | MMA 1 ... MMA 1 (x8) |
2207 //   | warp_QP 2 | MMA 2 ... MMA 2 (x8) |
2208 //   | warp_QP 3 | MMA 3 ... MMA 3 (x8) |
2209 // tile_V_Bc selects which 16-column slice of P to use:
2210 //   tile_V_Bc=0 → cols 0:16 → MMA indices 0,1 → w=0
2211 //   tile_V_Bc=1 → cols 16:32 → MMA indices 2,3 → w=2
2212 //   tile_V_Bc=2 → cols 32:48 → MMA indices 4,5 → w=4
2213 //   tile_V_Bc=3 → cols 48:64 → MMA indices 6,7 → w=6
2214 // 对应的 MMA A fragment 布局可以这样记:
2215 //   - rows 0~7: lane 0~3 → {a0,a1} 与 {a4,a5}, lane 4~7 → 下一行, 依次类推
2216 //   - rows 8~15: lane 0~3 → {a2,a3} 与 {a6,a7}, lane 4~7 → 下一行, 依次类推
2217 // 当前实现正是利用这一路径下 A fragment 与前面生成的 P fragment 可以直接对接,
2218 // 才能把 R_S 中的 P 直接喂给 HMMA16816 做 P@V; 复习时不要把它背成对所有 MMA
2219 // fragment 都无条件成立的通用结论。
2220 // layout转换逻辑:
2221 //   C layout: 8 x (16 x 8) layout → A layout: 4 x (16 x 16) layout
2222 int w = tile_V_Bc * 2;
2223 #pragma unroll
2224 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2225 #pragma unroll
2226 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2227     HMMA16816(R_0[i][j][0], R_0[i][j][1], // C fragment output
2228         R_S[i][w][0], R_S[i][w][1], R_S[i][w + 1][0], R_S[i][w + 1][1], // A fragment = P
2229         R_V[j][0], R_V[j][1], // B fragment = V
2230         R_0[i][j][0], R_0[i][j][1]); // C fragment output
2231     }
2232 }
2233 } // end for tile_V_Bc
2234 __syncthreads();
2235
2236 // =====
2237 // 3e: Online rescaling — 0_new = exp(m_old - m_new) * 0_old + P@V
2238 // =====
2239 // 公式来源: FA2 paper Eq.(7-8), 使用 exp(m_old - m_new) 做 0 与 1 的 rescale
2240 #pragma unroll

```

```

2241 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2242     float block_row_max_new_0 = lane_row_max_new[i][0];
2243     float block_row_max_new_1 = lane_row_max_new[i][1];
2244     float block_row_sum_new_0 = lane_row_sum_new[i][0];
2245     float block_row_sum_new_1 = lane_row_sum_new[i][1];
2246
2247     float block_row_max_old_0 = lane_block_row_max_old[i][0];
2248     float block_row_max_old_1 = lane_block_row_max_old[i][1];
2249
2250     block_row_max_new_0 = max(block_row_max_old_0, block_row_max_new_0);
2251     block_row_max_new_1 = max(block_row_max_old_1, block_row_max_new_1);
2252
2253     // Handle first iteration: m_old = -inf, need to use m_new directly
2254     block_row_max_old_0 =
2255         (tile_K_seqlen > 0 ? block_row_max_old_0 : block_row_max_new_0);
2256     block_row_max_old_1 =
2257         (tile_K_seqlen > 0 ? block_row_max_old_1 : block_row_max_new_1);
2258
2259     float rescale_o_factor_0 =
2260         __expf(block_row_max_old_0 - block_row_max_new_0);
2261     float rescale_o_factor_1 =
2262         __expf(block_row_max_old_1 - block_row_max_new_1);
2263
2264     // Rescale O_old + Add P@V in one fused step
2265     #pragma unroll
2266     for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2267         // R_0 / R_D 与前面的 R_S 一样, 都按 MMA C fragment 布局解释:
2268         // reg[0] -> rows 0~7 的 {c0,c1}
2269         // reg[1] -> rows 8~15 的 {c2,c3}
2270         float2 t_reg_0_0 = __half22float2(HALF2(R_0[i][j][0]));
2271         float2 t_reg_0_1 = __half22float2(HALF2(R_0[i][j][1]));
2272         float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2273         float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2274
2275         // O_new = exp(m_old - m_new) * O_old + P@V (fused multiply-add)
2276         t_reg_D_0.x = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.x, t_reg_0_0.x);
2277         t_reg_D_0.y = __fmaf_rn(rescale_o_factor_0, t_reg_D_0.y, t_reg_0_0.y);
2278         t_reg_D_1.x = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.x, t_reg_0_1.x);
2279         t_reg_D_1.y = __fmaf_rn(rescale_o_factor_1, t_reg_D_1.y, t_reg_0_1.y);
2280
2281         HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2282         HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2283     }
2284
2285     // Update l: l_new = exp(m_old - m_new) * l_old + row_sum(P)
2286     float block_row_sum_old_0 = lane_block_row_sum_old[i][0];
2287     float block_row_sum_old_1 = lane_block_row_sum_old[i][1];
2288     lane_block_row_sum_old[i][0] = __fmaf_rn(
2289         rescale_o_factor_0, block_row_sum_old_0, block_row_sum_new_0);
2290     lane_block_row_sum_old[i][1] = __fmaf_rn(
2291         rescale_o_factor_1, block_row_sum_old_1, block_row_sum_new_1);
2292
2293     // Update m
2294     lane_block_row_max_old[i][0] = block_row_max_new_0;
2295     lane_block_row_max_old[i][1] = block_row_max_new_1;
2296 }
2297
2298 // Wait for next K tile to be ready in smem before next iteration
2299 if constexpr (kStage > 1) {
2300     if ((tile_K_seqlen + 1) < Tc) {
2301         CP_ASYNC_WAIT_GROUP(0);
2302     }
2303     __syncthreads();
2304 }
2305 // end outer loop over K seqlen
2306 __syncthreads();
2307
2308 // =====
2309 // Step 4: 最终 rescale — O_final = (1/l_final) * O_final
2310 // =====

```

```

2311 #pragma unroll
2312 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2313     float rescale_factor_0 = __frcp_rn(lane_block_row_sum_old[i][0]);
2314     float rescale_factor_1 = __frcp_rn(lane_block_row_sum_old[i][1]);
2315 #pragma unroll
2316 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2317     float2 t_reg_D_0 = __half22float2(HALF2(R_D[i][j][0]));
2318     float2 t_reg_D_1 = __half22float2(HALF2(R_D[i][j][1]));
2319     t_reg_D_0.x = rescale_factor_0 * t_reg_D_0.x;
2320     t_reg_D_0.y = rescale_factor_0 * t_reg_D_0.y;
2321     t_reg_D_1.x = rescale_factor_1 * t_reg_D_1.x;
2322     t_reg_D_1.y = rescale_factor_1 * t_reg_D_1.y;
2323     HALF2(R_D[i][j][0]) = __float22half2_rn(t_reg_D_0);
2324     HALF2(R_D[i][j][1]) = __float22half2_rn(t_reg_D_1);
2325 }
2326 }
2327
2328 // =====
2329 // Step 5: Epilogue — Collective store via warp shuffle + 128-bit store
2330 // =====
2331 // 利用 warp shuffle 将分散在各 lane 的寄存器数据收集到 lane 0~3,
2332 // 然后用 LDST128BITS (st.global.v4.f32) 一次性写入 16 bytes
2333 #pragma unroll
2334 for (int i = 0; i < kWarpTileSeqLenP; ++i) {
2335 #pragma unroll
2336 for (int j = 0; j < kWarpTileHeadDimV; ++j) {
2337     uint32_t R_Z[2][4];
2338     R_Z[0][0] = R_D[i][j][0];
2339     R_Z[1][0] = R_D[i][j][1];
2340     R_Z[0][1] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 1, 4);
2341     R_Z[0][2] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 2, 4);
2342     R_Z[0][3] = __shfl_sync(0xffffffff, R_D[i][j][0], lane_id + 3, 4);
2343     R_Z[1][1] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 1, 4);
2344     R_Z[1][2] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 2, 4);
2345     R_Z[1][3] = __shfl_sync(0xffffffff, R_D[i][j][1], lane_id + 3, 4);
2346
2347     // st.global.v4.f32: 128-bit store, 4 lanes × 32-bit
2348     if (lane_id % 4 == 0) {
2349         int store_warp_regs_0_Br =
2350             warp_QP * (kMmaAtomM * kWarpTileSeqLenP) + i * kMmaAtomM;
2351         int store_lane_gmem_0_Br =
2352             Q_tile_id * Br + store_warp_regs_0_Br + lane_id / 4;
2353         int store_warp_regs_0_d =
2354             warp_KV * (kMmaAtomN * kWarpTileHeadDimV) + j * kMmaAtomN;
2355         int store_lane_gmem_0_d = store_warp_regs_0_d;
2356         int store_gmem_0_addr_0 = 0_gmem_offset +
2357             (store_lane_gmem_0_Br + 0) * kHeadDim +
2358             store_lane_gmem_0_d;
2359         int store_gmem_0_addr_1 = 0_gmem_offset +
2360             (store_lane_gmem_0_Br + 8) * kHeadDim +
2361             store_lane_gmem_0_d;
2362         // LDST128BITS = reinterpret_cast<float4*>
2363         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_0]) =
2364             *reinterpret_cast<float4*>(&R_Z[0][0]);
2365         *reinterpret_cast<float4*>(&0[store_gmem_0_addr_1]) =
2366             *reinterpret_cast<float4*>(&R_Z[1][0]);
2367     }
2368 }
2369 }
2370 }
2371
2372 // =====
2373 // End of notes-v2.cu
2374 // =====
2375 // 本文件覆盖了面试中最高频的 CUDA kernel 考点 (26 个 kernel) :
2376 // * 基础原语: warp_reduce (O(logN) butterfly), block_reduce (两级:
2377 //   warp-smem→warp0) * 优化手段: coalescing, tiling, thread tile, vectorize,
2378 //   pipeline, tensor core * Softmax 递进: naive → safe(2-pass) → online(1-pass,
2379 //   FA 基础) * GEMM 五层金字塔: tiling → thread tile → vectorize → MMA(TN布局)
2380 //   → WGMMMA(warp spec) * FlashAttention: split-Q + online softmax + R_S

```

```
2381 // 寄存器复用 P@V ★ Bank Conflict: 原理 + PAD 解决方案 + 四步演进 ★ Memory
2382 // Hierarchy: HBM → L2 → L1/SMEM → Register ★ BLAS 布局约定:
2383 // N=col-major(Normal), T=row-major(Transposed), TN=A行B列
2384 // =====
```